

GoF 23가지

게임 프로그래밍

디자인 패턴

목차

1. 서론

객체 지향 프로그래밍 개요

객체 지향 프로그래밍 개념

객체 지향 - UML

클래스의 종류

2. 디자인 패턴

디자인 패턴 개요

디자인 패턴 종류 (GoF)

└ 디자인 패턴 - 생성

└ 디자인 패턴 - 구조

└ 디자인 패턴 - 행동

3. 참고 문헌

※ 전개 과정

패턴 설명

예제 설명

※ 파일 다운로드

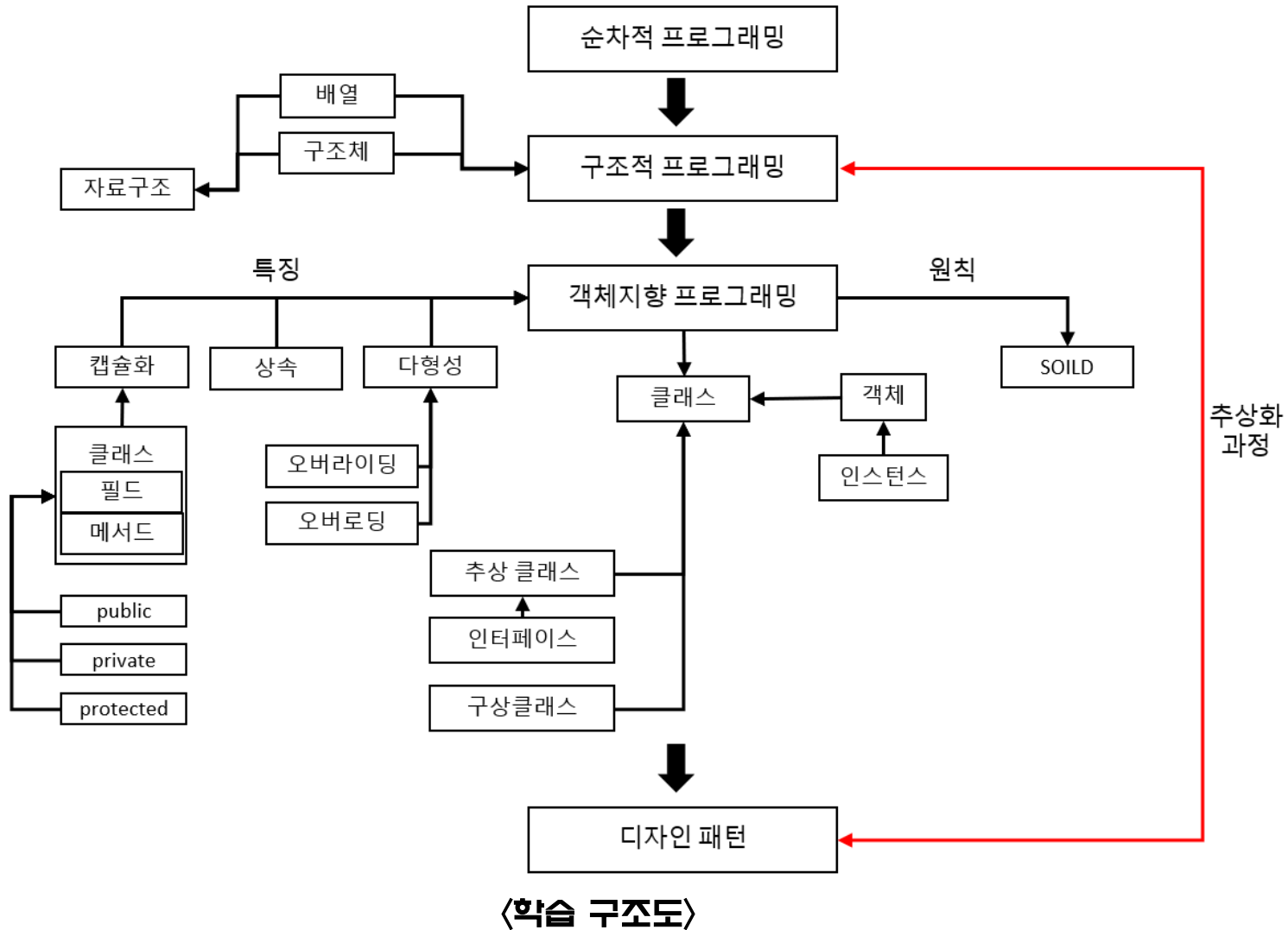
<https://github.com/TakeKieer/DesignPatten>

구글 드라이브 – 참고문헌

└ main.cpp 정의

서론

객체 지향 프로그래밍



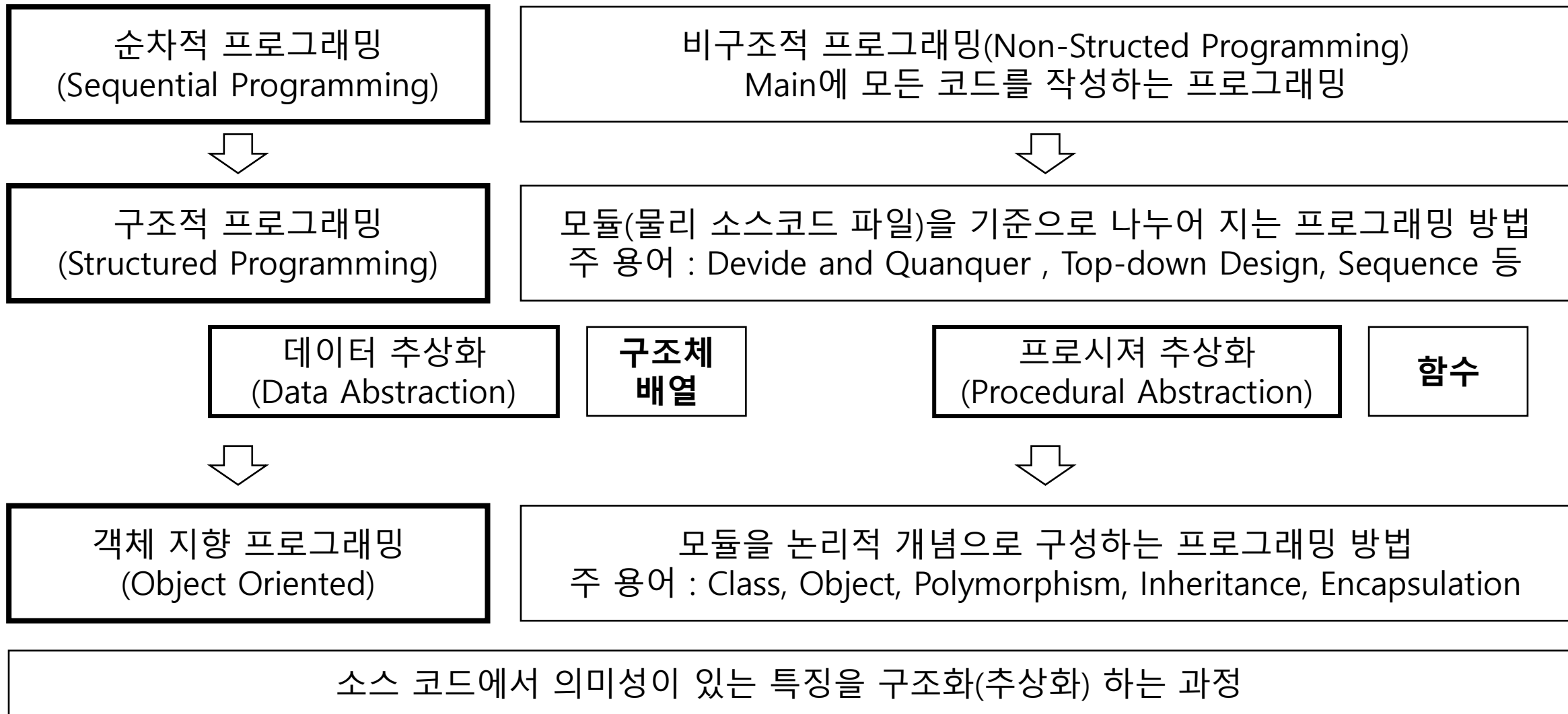
목적

1. 왜? → 어떻게? 방식으로 학습하였을 때, “이러한 점이 편했구나”를 알아가게 되면서 지식을 습득한다.

2. 프로그래밍을 확장하거나 관리하는 방법을 생각하기 위한 기본 지식을 습득한다.

서론

알아 두어야 할 개념



객체 지향 프로그래밍 (Object Oriented) 특징과 SOLID

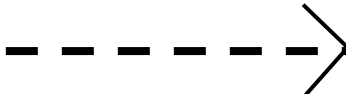
캡슐화 : 필드, 메소드를 묶어 외부로부터 코드 보호

상속 : 기존 클래스로부터 코드를 공유

다형성 : 하나의 타입에 여러 객체를 대입

- 1) SRP (Single Responsibility Principle, 단일 책임 원칙) : 클래스나 함수는 단 하나의 기능만을 가진다.
- 2) OCP (Open-Close Principle, 개방-폐쇄 원칙) : 자주 변경될 수 있는 내용을 수정하기 쉽게 설계한다.
- 3) LCP (Liskov Substitution Principle, 리스코프 치환 원칙) : 파생 클래스는 올바른 상속 관계를 맺는다.
- 4) ISP (Interface Segregation Principle, 인터페이스 분리 원칙) : 이용하지 않은 메서드에 의존하지 않는다.
- 5) DIP (Dependency Inversion Principle, 의존 역전 원칙) : 의존 관계는 변하기 어려운 것이어야 한다.

객체 지향 프로그래밍 (Object Oriented) 관계 – UML 상관 표

Generalization (일반화)	상속	
Realization (실체화)	특정 행동을 인터페이스화 하여 연관을 맺는 관계	
Association (연관)	한 객체가 다른 객체를 소유하거나 파라미터로 객체를 받아서 처리하는 관계	
Directed Association (직접 연관)	한 객체가 다른 객체를 직접적으로 처리하는 관계	
Dependency (의존)	한 객체가 다른 객체를 소유하지는 않지만, 다른 객체의 변경에 영향을 받는 관계	
Composition (구성)	여러 객체가 한 객체로부터 파생된 관계	
Aggregation (집합)	여러 객체가 한 객체와 연관된 관계 합성 / 집약	

클래스의 종류

추상 클래스 (abstract)

- 순수 가상 함수(추상 메소드)를 가지고 있는 클래스
- 다른 형식의 클래스로만 접근이 가능한 객체를 생성할 수 없는 클래스

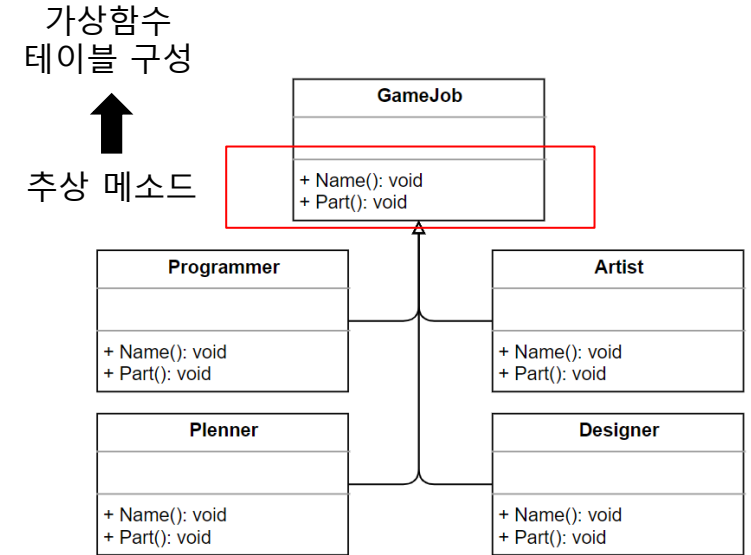
인터페이스 (Interface)

- 특정 기능을 구현하기로 약속된 형식
- 추상 클래스의 극단적 형태, 메소드 선언만 가능하다.

C++	struct 타입으로 선언 이후 상속으로 선언가능 (class FunctionClass : IClassInterface)
JAVA	Interface 타입으로 선언 상속으로 선언가능 (class FClass implement IClass)

구상 클래스 (구현, 구체 = Concrete)

- 추상 클래스와 인터페이스에서 상속 받은 메소드를 실행 가능한 클래스
- 추상 클래스가 아닌 모든 클래스를 구상 클래스라 정의한다.



<추상 클래스 예시>

```
class CClass : AClass, IClass
```

디자인 패턴

디자인 패턴이란

디자인 패턴

프로그램을 개발하는 과정에서 빈번하게 발생하는 디자인 상의 문제를 상황에 따라 간편하게 적용해서 쓸 수 있는 상태로 만든 패턴의 형태를 의미한다.

- 소프트웨어 구조를 구성하기 위한 일련의 표준

디자인 패턴 목표

- 프로젝트 개발 기간 동안 코드를 쉽게 이해할 수 있도록 구조를 깔끔하게 만든다.
- 실행 성능을 최적화 한다.
- 지금 개발 중인 기능을 최대한 빠르게 구현한다.

디자인 패턴

디자인 패턴의 23가지 종류

생성
(Creational Patten)

구조
(Structural Pattern)

행동
(Behavioral Pattern)

1. 추상 팩토리 (Abstract Factory)

6. 어댑터 (Adapter)

클래스
중심적

13. 인터프리터 (Interpreter)
14. 템플릿 메서드 (Template Method)

객체 및 인스턴스
중심적

2. 빌더 (Builder)
3. 팩토리 메서드 (Factory Method)
4. 프로토타입 (Prototype)
5. 싱글톤 (singleton)

7. 브리지(Bridge) - 다리
8. 컴포지드(Composite) - 합성
9. 데코레이터(Decorator) - 구성
10. 퍼사드(Façade) - 외관
11. 플라이웨이트(Flyweight) - 공유
12. 프록시(Proxy) - 임시

15. 책임연쇄 (Chain of Responsibility)
16. 명령 (Command)
17. 이터레이터 (Iterator)
18. 미디에이터 (Mediator)
19. 메멘토 (Memento)
20. 관찰자 (Observer)
21. 전략(Strategy)
22. 상태(State)
23. 방문자(Visitor)

디자인 패턴

디자인 패턴의 23가지 종류

	디자인 패턴	예제 프로그램	페이지
생성 패턴	팩토리 메소드	게임 제작을 지원하는 프로그램	p. 12
	추상 팩토리		p. 18
	빌더		p. 24
	프로토 타입	게임을 사전에서 생성 복제하여 설명하는 프로그램	p. 28
	싱글톤베이스	지도자가 명령을 내리는 프로그램	p. 32
구조 패턴	어댑터 패턴	액션 게임의 기능을 가지고 있는 퍼즐게임	p. 39
	브릿지 패턴	사용자가 자신의 프로그래머 능력을 확인하는 프로그램	p. 42
	컴포지드 패턴	가입자들에게 명령을 내리는 프로그램	p. 45
	데코레이터 패턴	중족 생성 이후 그 중족에서 속성을 부여하기 위한 프로그램	p. 50
	퍼사드 패턴	게임 제작에 필요한 기능들을 모아 놓은 사전	p. 53
	플라이웨이트 패턴	타일 정보 공유 플라이 웨이트 프로그램	p. 57
	프록시 패턴	캐릭터에 대한 정보를 불러오는 프로그램	p. 61
행동 패턴			p. 65

디자인 패턴

생성 패턴 개요

[목록으로](#)

클래스와 객체, 인스턴스

상위개념

클래스 : 객체를 만들어 내기 위한 설계

객체 : 소프트웨어 세계에 구현할 대상(클래스 인스턴스)

인스턴스 : 소프트웨어에 구현된 구체적인 실체

생성 패턴 (Creation Pattern)

인스턴스를 『만드는 절차』를 『추상화』하는 패턴

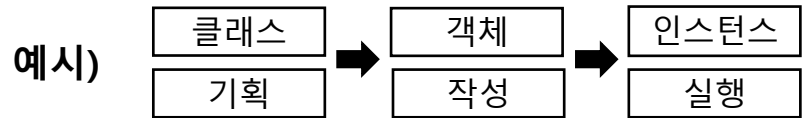
- 클래스 생성 패턴 : 인스턴스로 만들 클래스를 다양하게 만들기 위한 용도
- 객체 생성 패턴 : 인스턴스화 작업을 다른 객체에서 생성

특징 ※ 동적할당(new) 키워드를 사용하지 않고 객체를 생성하는 유동적인 방법

- 시스템이 어떤 구상(concrete) 클래스를 사용하는 지에 대한 정보를 캡슐화
- 클래스의 인스턴스들의 생성, 연결 부분에 대한 정보를 완전히 분리 및 은폐 (생성에 대한 유연성 확보)

고려 사항

- 생성 패턴으로 분류된 패턴들은 상호 보완적일 수 있으며 상호 경쟁적일 수 있다.
- 인스턴스를 생성, 연결 하기 위한 패턴을 판단하기 어렵다



생성 패턴 1 - 팩토리 메소드

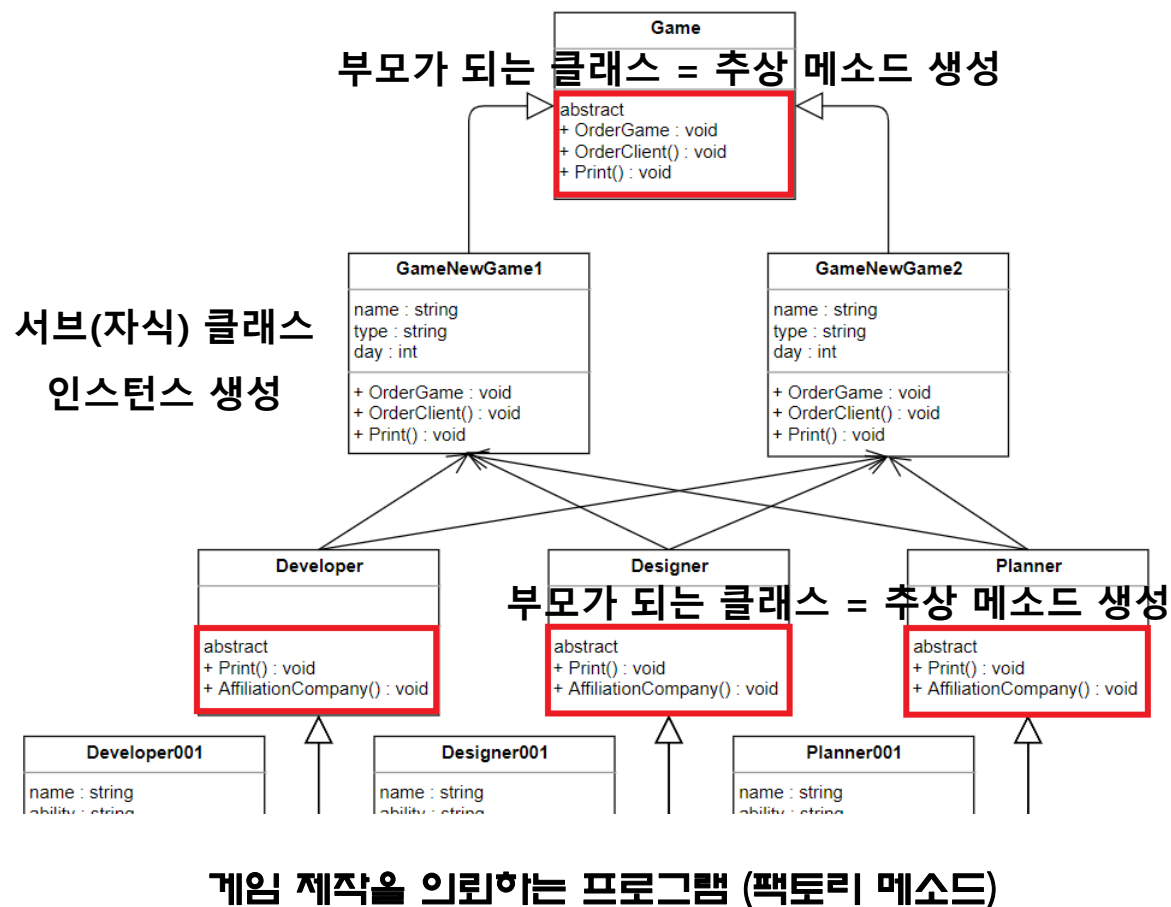
팩토리 메소드 (Factory Method)

객체를 생성하는 인터페이스 정의

- 서브 클래스에서 클래스의 인스턴스를 만든다.

팩토리 메소드 과정

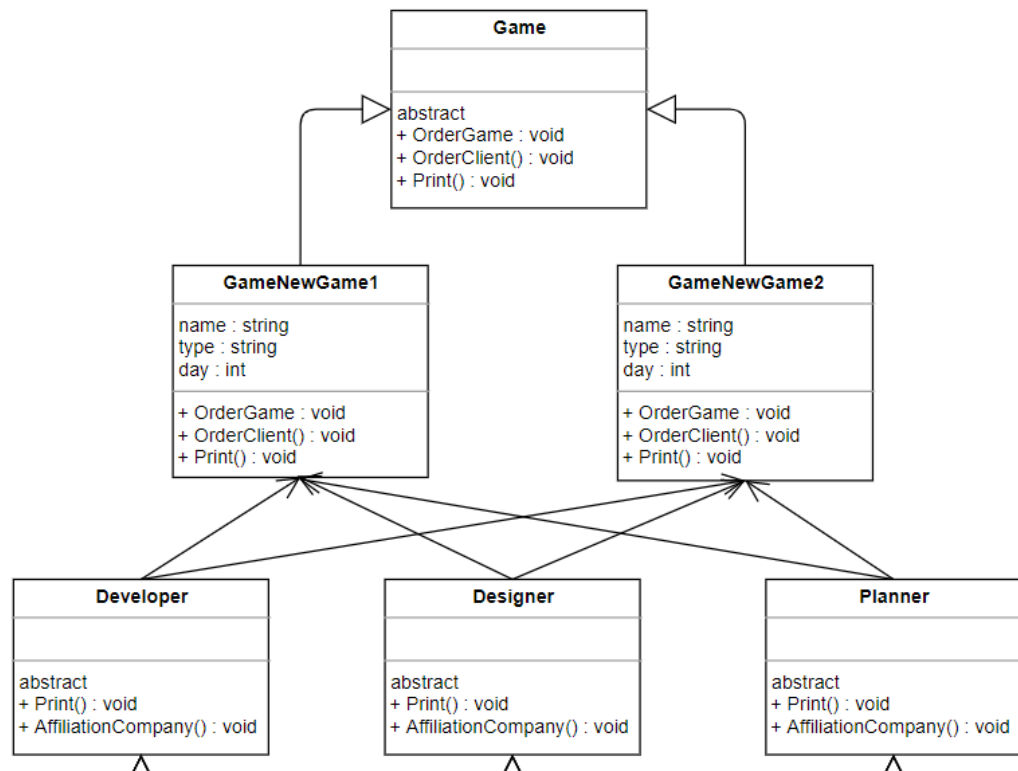
1. 객체를 생성하는 인터페이스 정의
2. 팩토리 생성, 팩토리 객체를 상속받아 서브클래스 구현
3. 서브 클래스에서 만든 메소드에 인스턴스 작성



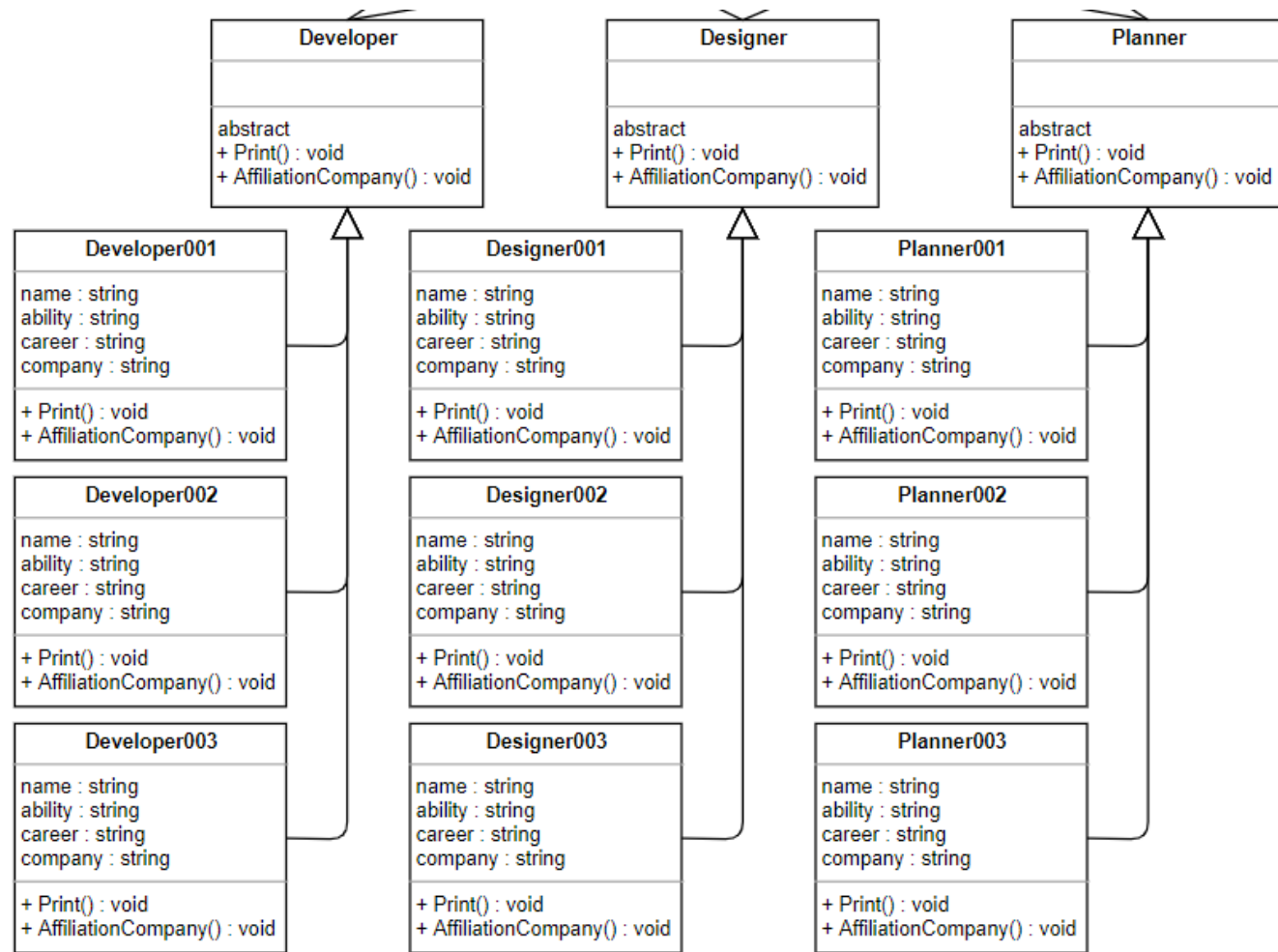
디자인 패턴

생성 패턴 1 – 팩토리 메소드

팩토리 메소드로 구현한 전체 UML



목록으로



디자인 패턴

[목록으로](#)

생성 패턴 1 – 팩토리 메소드

Main에서 호출



현실 세계에 맞는 추상화 과정을
보여준다

특징

1. 유연성과 확장성이 뛰어나다.
2. 인터페이스에서 객체 생성을 요청한다.

```
Game* game = new GameNewGame1();           // 첫번째 게임을 의뢰
game->OrderGame("스타크래프트", "전략", 60); // 게임 의뢰
game->Print();                               // 게임 의뢰 확인
game->OrderClient();                         // 게임 개발자 정보

delete game;
```

추상화 된 클래스 호출 내역 – 게임 제작 의뢰 정보

사용 효과

클래스 간의 결합도를 낮춘다.

- 클래스의 변경점이 생겼을 때 다른 클래스에 영향 ↓
- 서브 클래스의 메소드만 변경함으로써, 다른 클래스의 내용에는 영향을 받지 않는다.

추상 메소드의 내용을 인터페이스로 정의함으로써 가독성 ↑

디자인 패턴

생성 패턴 1 - 팩토리 메소드

Game 팩토리

목록으로

```
class Game {
public:
    // abstract Method 추상 메소드
    virtual void OrderGame(string _name, string _type, int _day) = 0; // 주문된 게임
    virtual void OrderClient() = 0; // 의뢰 대상
    virtual void Print() = 0; // 제작될 게임 정보 출력
};
```

부모 클래스 -> 추상 메소드만 구성한다 -> 인터페이스 정의



자식 클래스에서 인스턴스를 작성한다.

1. 만들 게임에 관한 클래스를 구성

2. 추상 메소드에 알맞게 개발 내역을 작성

Game : 추상 메소드

Game1 메소드 : 기능 인스턴스

Game2 메소드 : 기능 인스턴스



하위 팩토리 사용도 같은 개념으로 작성

```
class GameNewGame1 : public Game
{
    string type;
    string name;
    int day;
public:
    // 게임 주문
    void OrderGame(string _name, string _type, int _day) {
        name = _name;
        type = _type;
        day = _day;
    }
    // 게임 정보
    void Print() {
        cout << "1 번째 게임" << endl;
        cout << "제목 : " << name << endl;
        cout << "장르 : " << type << endl;
        cout << "일자 : " << day << endl;
    }
    // 클라이언트
    void OrderClient() {
        Developer* developer = new Developer001();
        Designer* designer = new Designer001();
        Planner* planner = new Planner001();

        developer->Print();
        designer->Print();
        planner->Print();
    }
};
```

```
class GameNewGame2 : public Game
{
    string type;
    string name;
    int day;
public:
    void OrderGame(string _name, string _type, int _day) {
        name = _name;
        type = _type;
        day = _day;
    }
    // 게임 정보
    void Print() {
        cout << "2 번째 게임" << endl;
        cout << "제목 : " << name << endl;
        cout << "장르 : " << type << endl;
        cout << "일자 : " << day << endl;
    }
    void OrderClient() {
        Developer* developer = new Developer002();
        Designer* designer = new Designer003();
        Planner* planner = new Planner003();

        developer->Print();
        designer->Print();
        planner->Print();
    }
};
```

디자인 패턴

생성 패턴 1 – 팩토리 메소드

목록으로

개발자 관련 팩토리

인터페이스

인스턴스 정의

```
class Planner {
public:
    // abstract Method 추상 메소드
    virtual void Print() = 0;
    virtual void AffiliationCompany() = 0;
};
```

```
class Planner001 : public Planner {
    string name = "AAA";
    string ability = "포토샵";    // 능력
    string career = "1년차";    // 경력
    string company = "KGA";

    void Print(){
        cout << "기획자001 : " << name << endl;
        cout << "주 능력 : " << ability << endl;
        cout << "경력 : " << career << endl;
        cout << "회사 : " << company << endl;
    }

    void AffiliationCompany(){
        cout << "소속 회사 : " << company << endl;
    }
};
```

```
class Planner002 : public Planner {
    string name = "BBB";
    string ability = "JAVA";    // 능력
    string career = "5년차";    // 경력
    string company = "GLOBAL";

    void Print(){
        cout << "기획자002 : " << name << endl;
        cout << "주 능력 : " << ability << endl;
        cout << "경력 : " << career << endl;
        cout << "회사 : " << company << endl;
    }

    void AffiliationCompany(){
        cout << "소속 회사 : " << company << endl;
    }
};
```

```
class Developer {
public:
    // abstract Method 추상 메소드
    virtual void Print() = 0;
    virtual void AffiliationCompany() = 0;
};
```

```
class Developer001 : public Developer {
    string name = "김철수";
    string ability = "C++";    // 능력
    string career = "1년차";    // 경력
    string company = "KGA";

    void Print(){
        cout << "개발자001 : " << name << endl;
        cout << "주 능력 : " << ability << endl;
        cout << "경력 : " << career << endl;
        cout << "회사 : " << company << endl;
    }

    void AffiliationCompany(){
        cout << "소속 회사 : " << company << endl;
    }
};
```

```
class Developer002 : public Developer {
    string name = "홍길동";
    string ability = "JAVA";    // 능력
    string career = "5년차";    // 경력
    string company = "NC";

    void Print(){
        cout << "개발자002 : " << name << endl;
        cout << "주 능력 : " << ability << endl;
        cout << "경력 : " << career << endl;
        cout << "회사 : " << company << endl;
    }

    void AffiliationCompany(){
        cout << "소속 회사 : " << company << endl;
    }
};
```

```
class Designer {
public:
    // abstract Method 추상 메소드
    virtual void Print() = 0;
    virtual void AffiliationCompany() = 0;
};
```

```
class Designer001 : public Designer {
    string name = "김영희";
    string ability = "포토샵";    // 능력
    string career = "1년차";    // 경력
    string company = "KGA";

    void Print(){
        cout << "디자이너001 : " << name << endl;
        cout << "주 능력 : " << ability << endl;
        cout << "경력 : " << career << endl;
        cout << "회사 : " << company << endl;
    }

    void AffiliationCompany(){
        cout << "소속 회사 : " << company << endl;
    }
};
```

```
class Designer002 : public Designer {
    string name = "안길동";
    string ability = "JAVA";    // 능력
    string career = "5년차";    // 경력
    string company = "NEXON";

    void Print(){
        cout << "디자이너002 : " << name << endl;
        cout << "주 능력 : " << ability << endl;
        cout << "경력 : " << career << endl;
        cout << "회사 : " << company << endl;
    }

    void AffiliationCompany(){
        cout << "소속 회사 : " << company << endl;
    }
};
```


디자인 패턴

[목록으로](#)

생성 패턴 1 - 팩토리 메소드



디자인 패턴

목차로

생성 패턴 2 - 추상 팩토리

추상 팩토리(Abstract Factory)

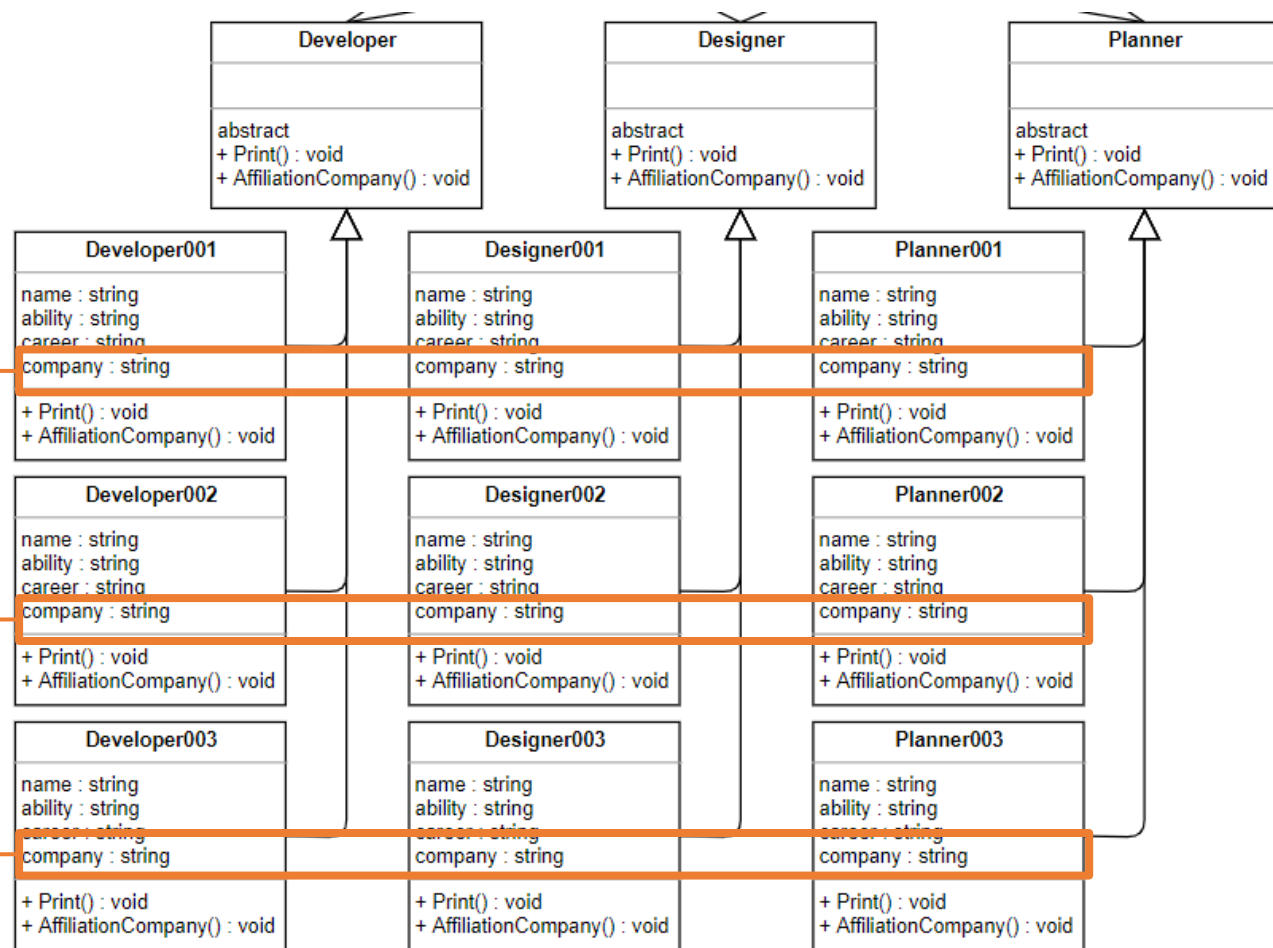
클래스를 특정 그룹으로 묶어서 관리하는 패턴

- 서브 클래스를 특정 그룹 묶어 관리

추상 팩토리 과정

1. 팩토리 생성 (팩토리 메소드)
2. 그룹 구분 변수 내역 지정
3. 그룹에 대한 추상 클래스 생성
4. 그룹에 대한 인스턴스 생성

회사 명으로
개발 인력
관리 가능



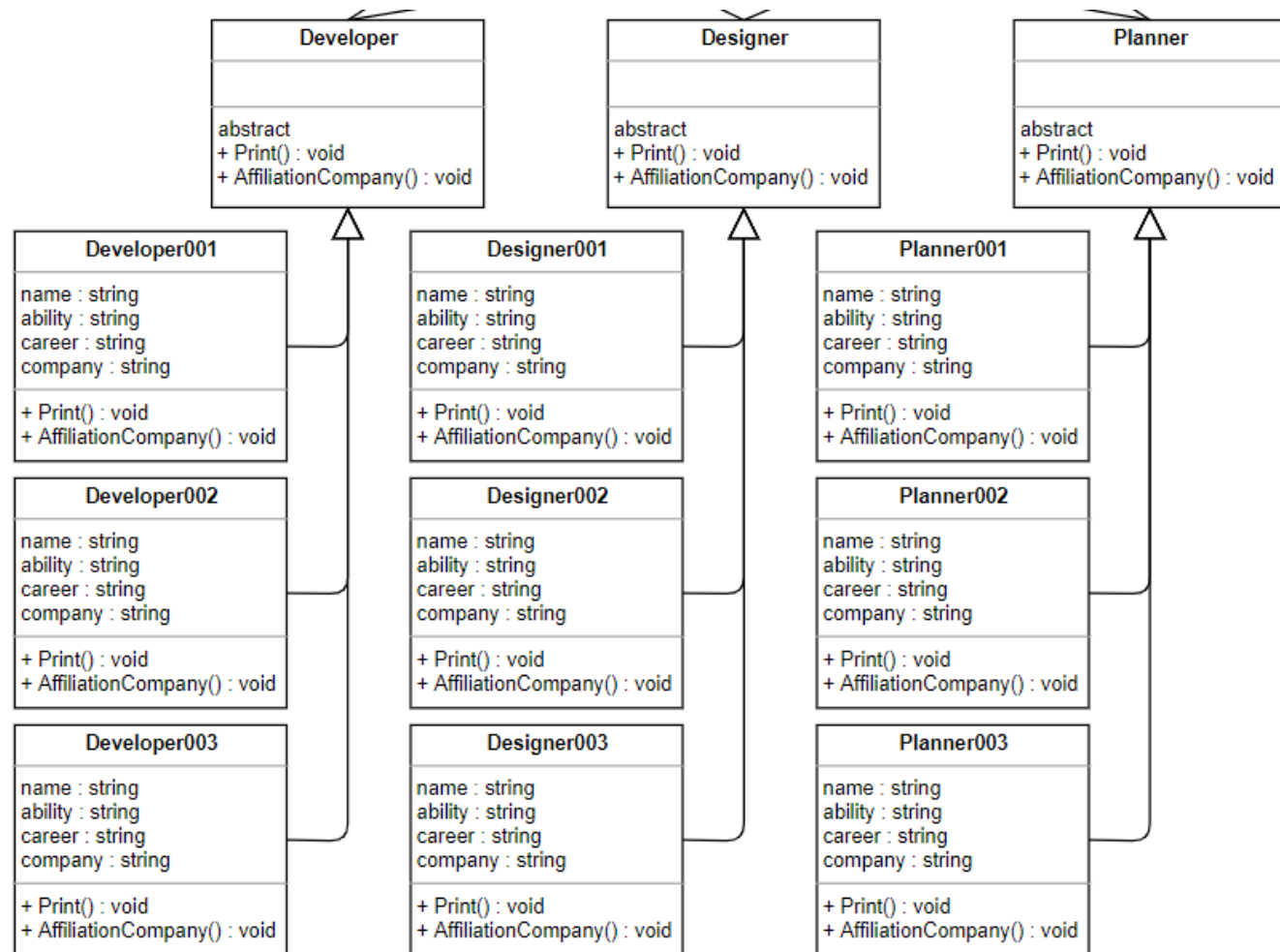
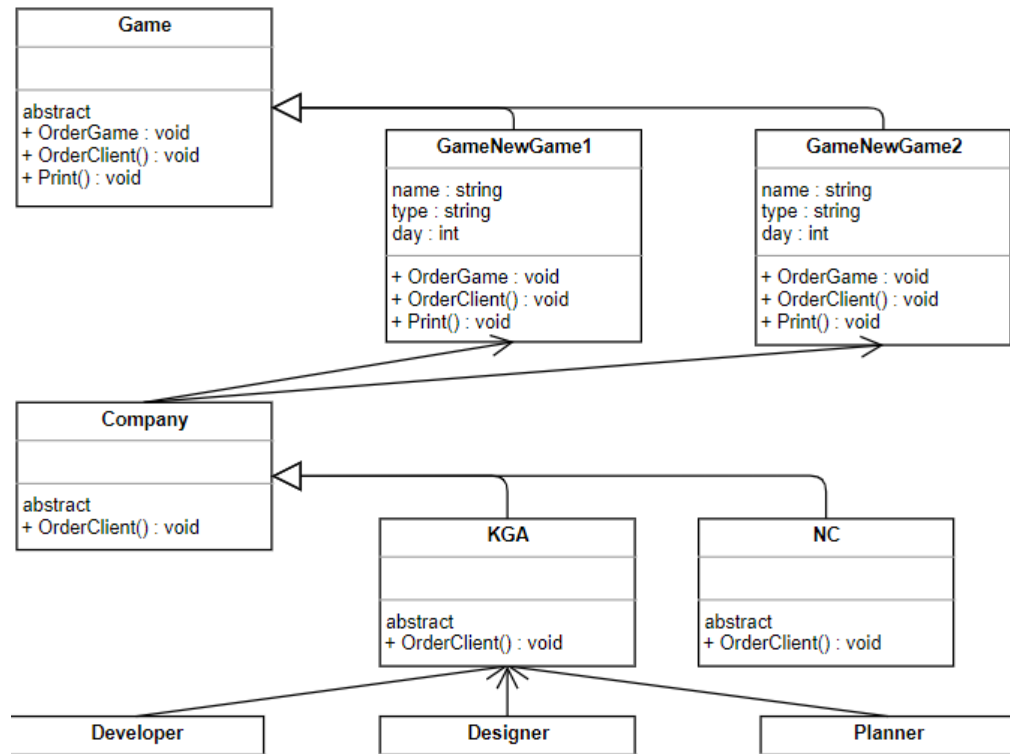
게임 제작을 지원하는 프로그램 (추상 팩토리)

디자인 패턴

목록으로

생성 패턴 2 – 추상 팩토리

추상 팩토리 UML



디자인 패턴

[목록으로](#)

생성 패턴 2 – 추상 팩토리

Main에서 호출



팩토리 메소드와 동일

```
Game* game = new GameNewGame1();           // 첫번째 게임을 의뢰
game->OrderGame("스타크래프트", "전략", 60); // 게임 의뢰
game->Print();                               // 게임 의뢰 확인
game->OrderClient();                          // 게임 개발자 정보

delete game;
```

추상화 된 클래스 호출 내역 – 게임 제작 의뢰 정보

특징

1. 유연성과 확장성이 뛰어나다.
2. 인터페이스에서 객체 생성을 요청한다.

사용 효과

- 팩토리 메소드와 동일
- 추가적으로 인스턴스들의 구조화 ↑ 용의성 ↑
- 인스턴스 확장, 축소 및 변경에 용의하다
- 객체로 관리하는 것이 아닌 **클래스**로 관리한다.

디자인 패턴

[목록으로](#)

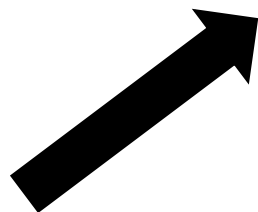
생성 패턴 2 – 추상 팩토리

Game 팩토리 -> 추상 메소드와 비교

```
void OrderClient() {  
    Developer* developer = new Developer001();  
    Designer* designer = new Designer001();  
    Planner* planner = new Planner001();  
  
    developer->Print();  
    designer->Print();  
    planner->Print();  
}
```

< 팩토리 메소드 >

KGA를 호출하면 KGA에 소속된 인력을
불러 올 수 있다



```
void OrderClient() {  
    Company* company = new KGA();  
  
    company->OrderClient();  
}
```

```
class Company  
{  
public:  
    virtual void OrderClient() = 0;  
};
```

```
class KGA : public Company  
{  
    void OrderClient()  
    {  
        Developer* developer = new Developer001();  
        Designer* designer = new Designer001();  
        Planner* planner = new Planner001();  
  
        developer->Print();  
        designer->Print();  
        planner->Print();  
    }  
};
```

< 추상 팩토리 >

디자인 패턴

생성 패턴 2 – 추상 팩토리

목록으로

이후 내역은 팩토리 메소드와 동일

인터페이스

인스턴스 정의

```
class Planner {
public:
    // abstract Method 추상 메소드
    virtual void Print() = 0;
    virtual void AffiliationCompany() = 0;
};
```

```
class Planner001 : public Planner {
    string name = "AAA";
    string ability = "포토샵";    // 능력
    string career = "1년차";    // 경력
    string company = "KGA";

    void Print(){
        cout << "기획자001 : " << name << endl;
        cout << "주 능력 : " << ability << endl;
        cout << "경력 : " << career << endl;
        cout << "회사 : " << company << endl;
    }

    void AffiliationCompany(){
        cout << "소속 회사 : " << company << endl;
    }
};
```

```
class Planner002 : public Planner {
    string name = "BBB";
    string ability = "JAVA";    // 능력
    string career = "5년차";    // 경력
    string company = "GLOBAL";

    void Print(){
        cout << "기획자002 : " << name << endl;
        cout << "주 능력 : " << ability << endl;
        cout << "경력 : " << career << endl;
        cout << "회사 : " << company << endl;
    }

    void AffiliationCompany(){
        cout << "소속 회사 : " << company << endl;
    }
};
```

```
class Developer {
public:
    // abstract Method 추상 메소드
    virtual void Print() = 0;
    virtual void AffiliationCompany() = 0;
};
```

```
class Developer001 : public Developer {
    string name = "김철수";
    string ability = "C++";    // 능력
    string career = "1년차";    // 경력
    string company = "KGA";

    void Print(){
        cout << "개발자001 : " << name << endl;
        cout << "주 능력 : " << ability << endl;
        cout << "경력 : " << career << endl;
        cout << "회사 : " << company << endl;
    }

    void AffiliationCompany(){
        cout << "소속 회사 : " << company << endl;
    }
};
```

```
class Developer002 : public Developer {
    string name = "홍길동";
    string ability = "JAVA";    // 능력
    string career = "5년차";    // 경력
    string company = "NC";

    void Print(){
        cout << "개발자002 : " << name << endl;
        cout << "주 능력 : " << ability << endl;
        cout << "경력 : " << career << endl;
        cout << "회사 : " << company << endl;
    }

    void AffiliationCompany(){
        cout << "소속 회사 : " << company << endl;
    }
};
```

```
class Designer {
public:
    // abstract Method 추상 메소드
    virtual void Print() = 0;
    virtual void AffiliationCompany() = 0;
};
```

```
class Designer001 : public Designer {
    string name = "김영희";
    string ability = "포토샵";    // 능력
    string career = "1년차";    // 경력
    string company = "KGA";

    void Print(){
        cout << "디자이너001 : " << name << endl;
        cout << "주 능력 : " << ability << endl;
        cout << "경력 : " << career << endl;
        cout << "회사 : " << company << endl;
    }

    void AffiliationCompany(){
        cout << "소속 회사 : " << company << endl;
    }
};
```

```
class Designer002 : public Designer {
    string name = "안길동";
    string ability = "JAVA";    // 능력
    string career = "5년차";    // 경력
    string company = "NEXON";

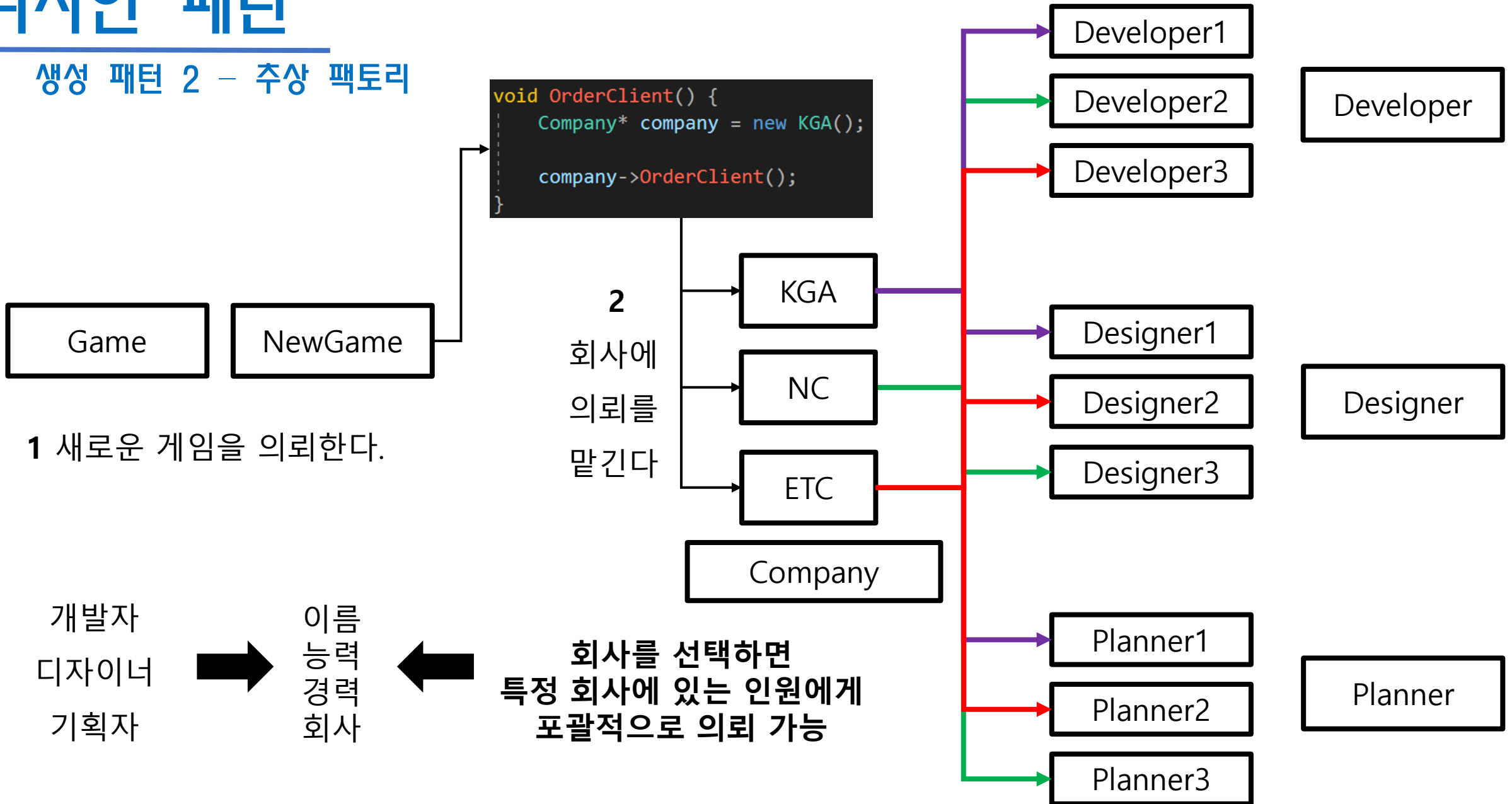
    void Print(){
        cout << "디자이너002 : " << name << endl;
        cout << "주 능력 : " << ability << endl;
        cout << "경력 : " << career << endl;
        cout << "회사 : " << company << endl;
    }

    void AffiliationCompany(){
        cout << "소속 회사 : " << company << endl;
    }
};
```

디자인 패턴

생성 패턴 2 - 추상 팩토리

[목록으로](#)



디자인 패턴

[목록으로](#)

생성 패턴 3 - 빌더

빌더 (Builder) 패턴

연산이 완료된 인스턴스를 제공할 수 있는 패턴

- 인스턴스의 내용과 인스턴스 생성과정을 분리한다.

추상 팩토리 : 각각 객체의 값을 그대로 제공

빌더 패턴 : 객체로 부터 완료된 결과 값 제공

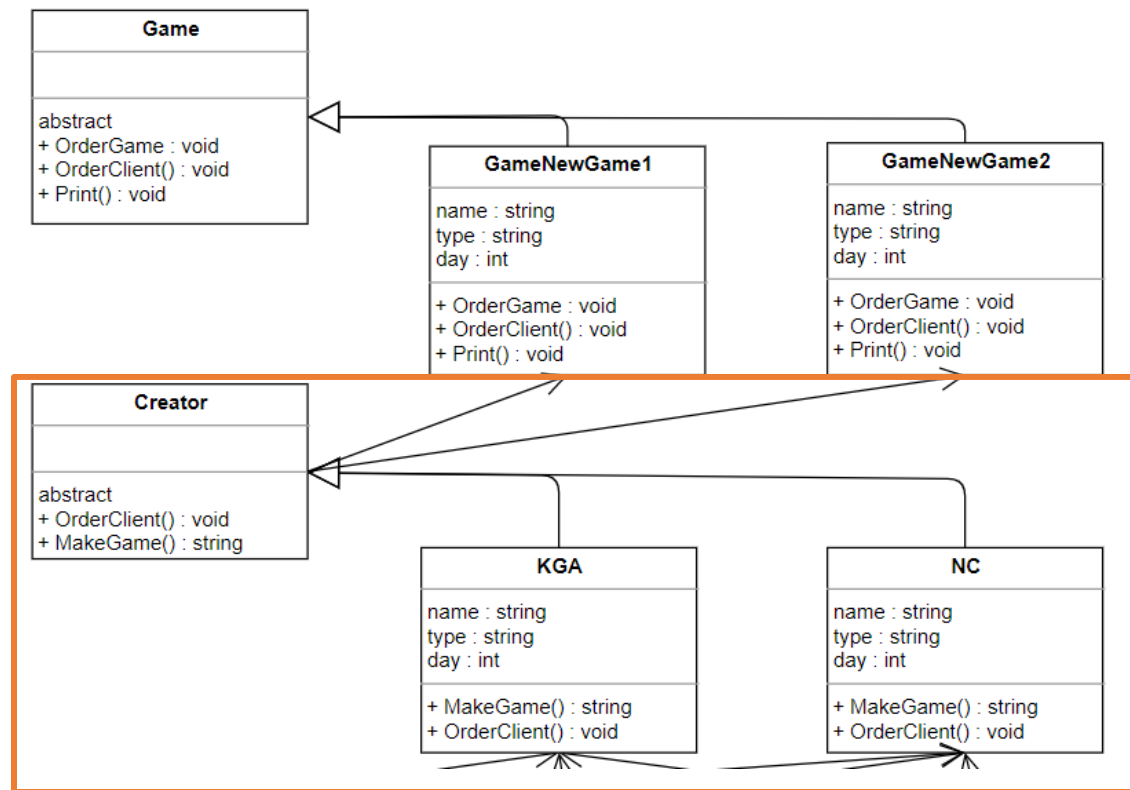
작업을 완료하는 메소드를 추가하는 과정을 의미한다.

예를 들면 다음과 같다



추상 팩토리 : 게임을 만드는데 사용되는 부품은?

빌더(Builder) 패턴 : 완성된 게임은?

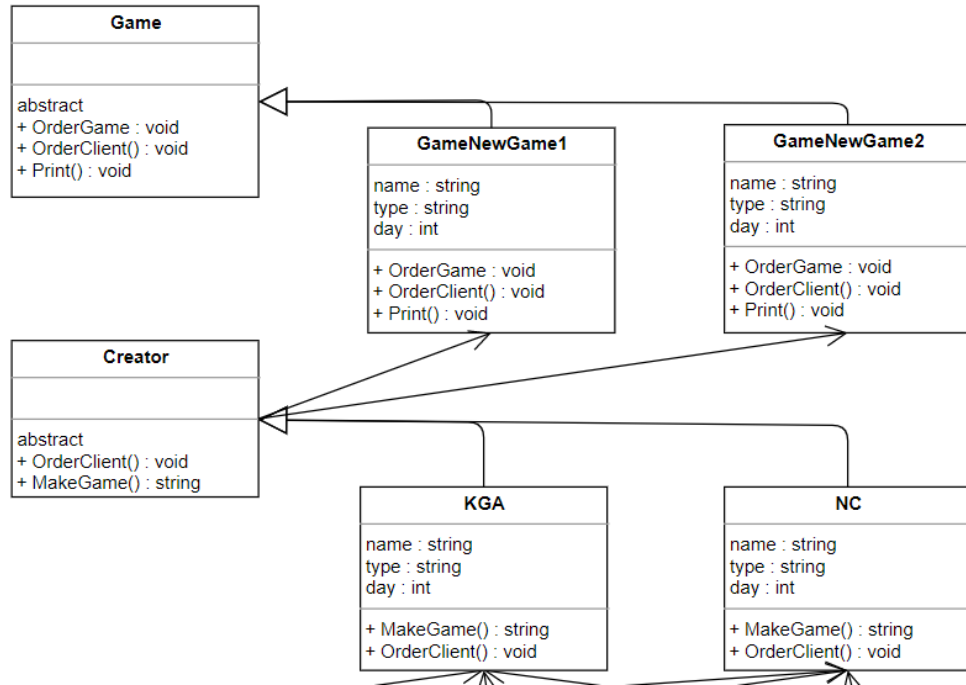


게임 제작을 지원하는 프로그램 (Builder 패턴)

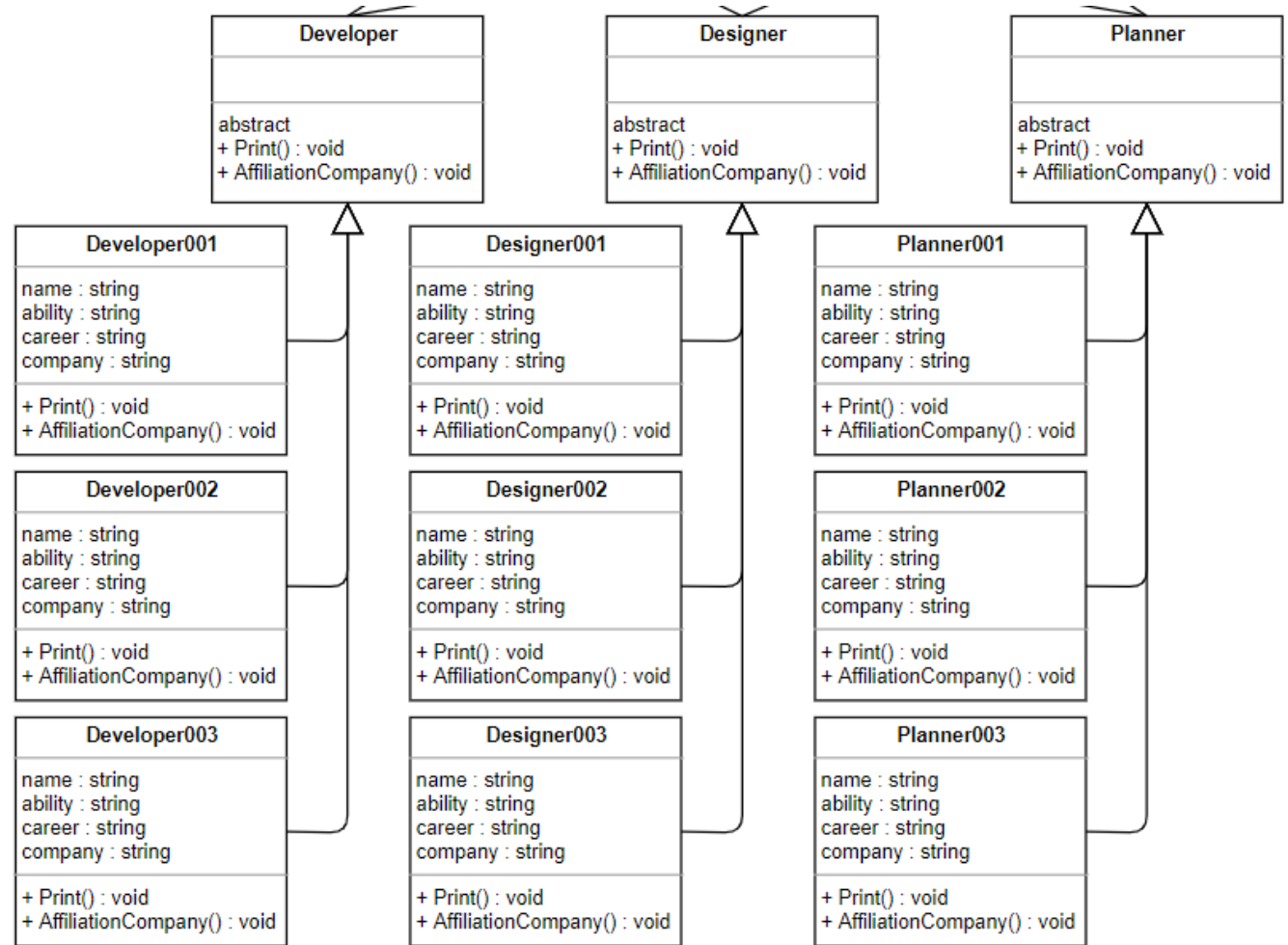
디자인 패턴

생성 패턴 3 - 빌더

Builder 패턴 UML



목록으로



디자인 패턴

[목록으로](#)

생성 패턴 3 - 빌더

```
class KGACreator : public Creator
{
    Designer* designer;
    Developer* developer;
    Planner* planner;
public:
    void OrderClient()
    {
        developer = new Developer001();
        designer = new Designer001();
        planner = new Planner001();
    }

    string MakeGame()
    {
        cout << planner->GetName() << " 기획자가 기획을 하고 있습니다." << endl;
        cout << developer->GetName() << " 개발자가 프로그래밍을 하고 있습니다." << endl;
        cout << designer->GetName() << " 디자이너가 디자인을 하고 있습니다." << endl;

        cout << "개발 결과를 갱신중...." << endl;
        string result = "KGA의 1번째 게임";
        return result;
    }
};
```

Company -> Creator 클래스로 변환 외

전체적으로 추상 클래스와 동일하다

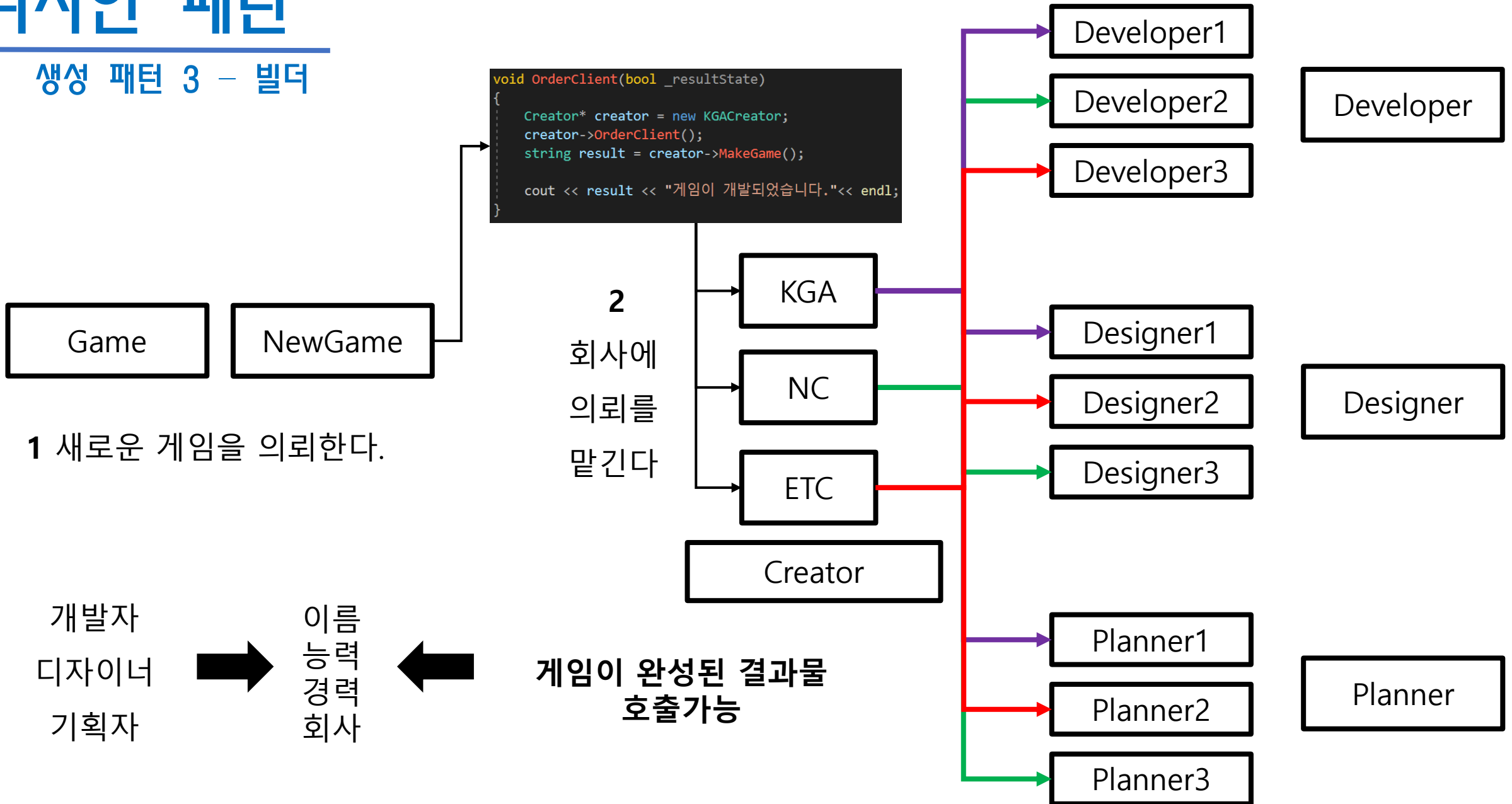
→ 클래스에서 작업자들을 설정한다

→ 인스턴스에서 게임이 만들어지는 일련의 결과물을 생성한다.

디자인 패턴

생성 패턴 3 - 빌더

[목록으로](#)



디자인 패턴

[목록으로](#)

생성 패턴 4 - 프로토타입

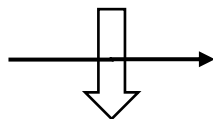
프로토 타입 패턴 (Prototype Patten)

객체를 복사해서 새로운 객체를 생성하는 패턴

- Spwan(GameType)을 통해 객체를 복사(생성)

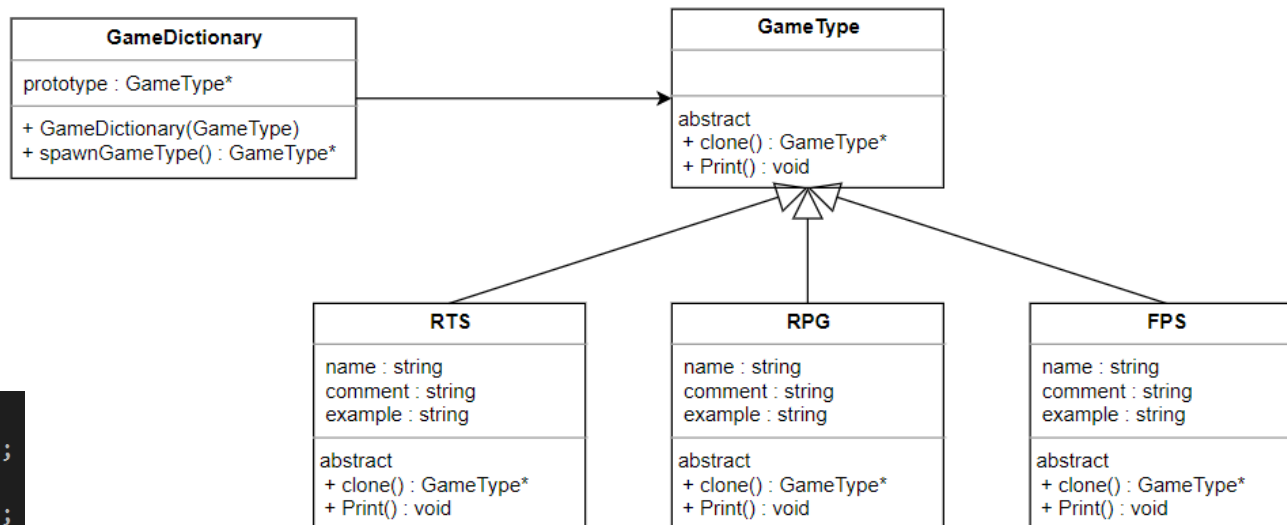
```
GameType* starcraft1 =  
    new RTS("스타크래프트1", "건물을 짓고, 유닛을 뽑고, 서로 싸우는 게임", "RTS");  
GameType* starcraft2 =  
    new RTS("스타크래프트2", "건물을 짓고, 유닛을 뽑고, 서로 싸우는 게임", "RTS");  
GameType* starcraft3 =  
    new RTS("스타크래프트3", "건물을 짓고, 유닛을 뽑고, 서로 싸우는 게임", "RTS");
```

각각의 클래스를
직접 생성



Spawn을 통해
상태 그대로 복사

```
GameType* starcraft =  
    new RTS("스타크래프트", "건물을 짓고, 유닛을 뽑고, 서로 싸우는 게임", "RTS");  
  
GameDictionary* dictionary = new GameDictionary(starcraft);  
  
GameType* starcraft1 = dictionary->spawnGameType();  
GameType* starcraft2 = dictionary->spawnGameType();  
GameType* starcraft3 = dictionary->spawnGameType();
```



게임을 설명하는 프로그램 UML (프로토타입 패턴)

같은 형태의 클래스를

각각의 클래스에 따라 직접 만들지 않고

Spawn을 통해 상태 그대로 복사한다

생성 패턴 4 – 프로토타입

Main에서 호출

```
int main()
{
    GameType* starcraft =
        new RTS("스타크래프트", "건물을 짓고, 유닛을 뽑고, 서로 싸우는 게임", "RTS");

    GameDictionary* dictionary = new GameDictionary(starcraft);

    GameType* starcraft1 = dictionary->spawnGameType();
    GameType* starcraft2 = dictionary->spawnGameType();
    GameType* starcraft3 = dictionary->spawnGameType();
}
```

1. 메인인 되는 클래스를 생성한다.

2. 사전에 데이터를 넣는다.

3. 넣은 데이터를 복사한다

```
starcraft1->Print();
starcraft2->Print();
starcraft3->Print();
```

내용이 잘 복사되었음을 확인할 수 있다

```
게임 제목 : 스타크래프트
게임 내용 : 건물을 짓고, 유닛을 뽑고, 서로 싸우는 게임
기타 내용 : RTS
게임 제목 : 스타크래프트
게임 내용 : 건물을 짓고, 유닛을 뽑고, 서로 싸우는 게임
기타 내용 : RTS
게임 제목 : 스타크래프트
게임 내용 : 건물을 짓고, 유닛을 뽑고, 서로 싸우는 게임
기타 내용 : RTS
```

디자인 패턴

생성 패턴 4 - 프로토타입

[목록으로](#)

```
class GameType
{
public:
    // 복제
    virtual GameType* clone() = 0;
    // 추상 메소드
    virtual void Print() = 0;
};
```

프로토 타입 패턴의 핵심은

자신을 할당할 수 있는 clone을 만드는 것이다

특징

- 객체를 있는 그대로 추가, 삭제가 용의하다.
- 새로운 객체를 명세하기 용의하다.
- 서브 클래스의 수를 줄일 수 있다.
- 주로 같은 종류의 아이템, 몬스터 등에 사용 가능하다.

```
class RTS : public GameType {
    string name;
    string comment;
    string example;
public:
    RTS(string _name, string _comment, string _example) :
        name(_name),
        comment(_comment),
        example(_example) {}

    virtual GameType* clone() { return new RTS(name, comment, example); }

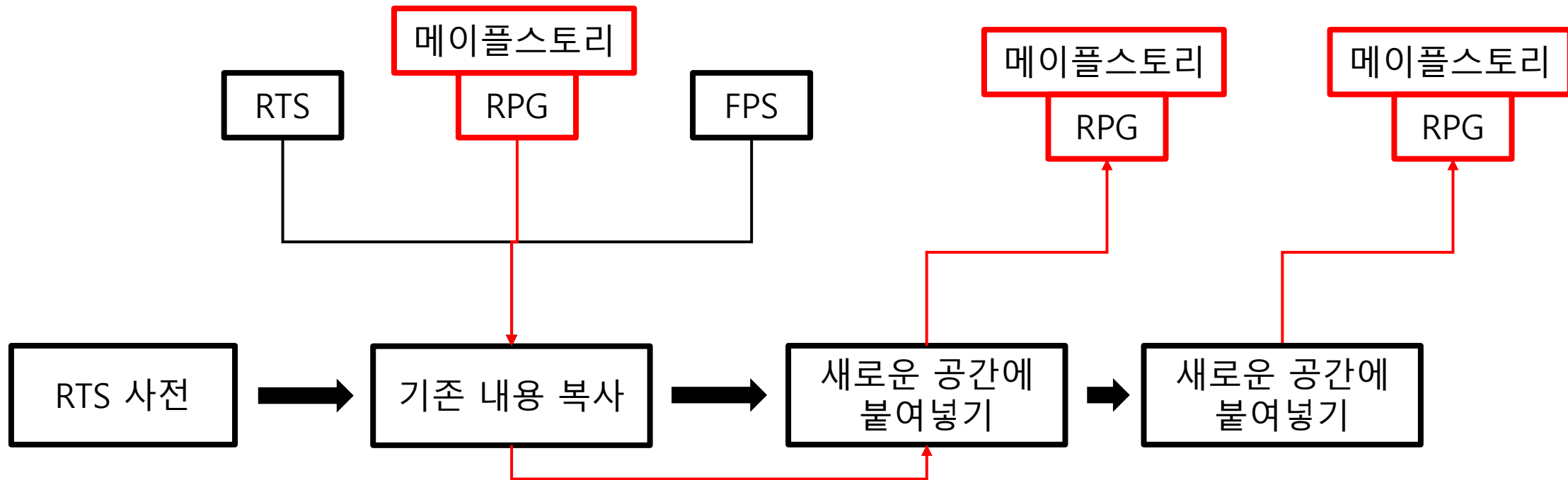
    void Print()
    {
        cout << "게임 제목 : " << name << endl;
        cout << "게임 내용 : " << comment << endl;
        cout << "기타 내용 : " << example << endl;
    }
};
```

```
class RPG : public GameType {
    string name;
    string comment;
    string example;
public:
    RPG(string _name, string _comment, string _example) :
        name(_name),
        comment(_comment),
        example(_example) {}

    virtual GameType* clone() { return new RPG(name, comment, example); }

    void Print()
    {
        cout << "게임 제목 : " << name << endl;
        cout << "게임 내용 : " << comment << endl;
        cout << "기타 내용 : " << example << endl;
    }
};
```

생성 패턴 4 – 프로토타입



우리가 자주 사용하는 복사 붙여넣기와 같은 형태를 띄고 있다.

상태 그대로를 붙여넣기 하기 때문에 포괄적인 작업에도 용의 할 수 있다.

생성 패턴 5 - 싱글톤

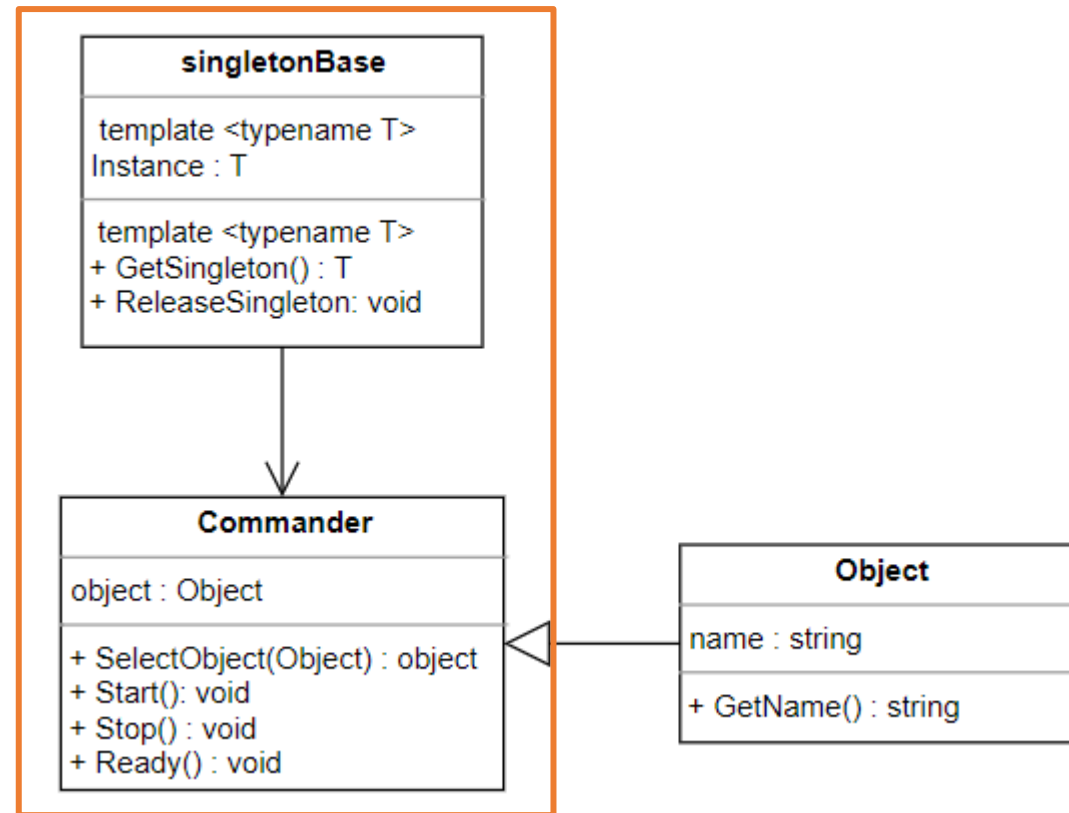
싱글톤 베이스(SingleTon)

전역에서 오직 하나의 인스턴스만을 가지는 패턴

- 공용 static의 기능들을 총괄하는 클래스에 사용

전역에서 사용된다 = 전역변수의 확장 개념

- 프로그램 어느 곳에서든 접근이 가능하므로 객체 지향과는 거리가 있는 패턴
- 적절한 제한적 사용이 요구되는 패턴



〈지도자가 명령을 내리는 프로그램〉

디자인 패턴

생성 패턴 5 - 싱글톤

싱글톤 베이스(Singleton) 형태 구성

```
template <typename T>
class singletonBase
{
protected:
    static T* instance;

    singletonBase() {};
    ~singletonBase() {};

public:
    static T* GetSingleton();
    void ReleaseSingleton();
};
```

목록으로

Inline 함수 : 함수를 프로그램 코드 안에 직접 삽입

- 기존 : 함수를 구성한 포인터로 전환 이후 함수 실행
- Inline : 함수 그대로를 실행 (속도가 빨라진다. 매크로와 비교된다.)

```
template<typename T>
T* singletonBase<T>::instance = 0;

template<typename T>
inline T* singletonBase<T>::GetSingleton()
{
    if (!instance) {
        instance = new T;
    }
    return instance;
}

template<typename T>
inline void singletonBase<T>::ReleaseSingleton()
{
    if (instance)
    {
        delete instance;
        instance = 0;
    }
}
```

디자인 패턴

생성 패턴 5 - 싱글톤

[목록으로](#)

싱글톤 베이스(SingleTon) 접목

```
#include "singletonBase.h"
#include "Common.h"
#include "Object.h"

class Commander : public singletonBase<Commander>
{
private:
    Object* object = NULL;
public:

    void SelectObject(Object* _object)
    {
        object = _object;
    }

    void Start() { ... }

    void Ready() { ... }

    void Stop() { ... }

    Commander() { object = NULL; }
};
```

Command를 하나의 인스턴스로 구성
전역으로 사용이 가능하다

어떤 다른 영역에서도 Commander가 명령을 내릴 수 있다

```
class Object
{
private:
    string name;
public:
    string GetName() { return name; }
    void SetName(string _name) { name = _name; }
};
```

생성 패턴 5 – 싱글톤

싱글톤 베이스(SingleTon) 사용

```
#include "Commander.h"
#include "Object.h"

#define COMMANDER Commander::GetSingleton()

int main()
{
    Object* human1 = new Object();
    human1->SetName("사람1");

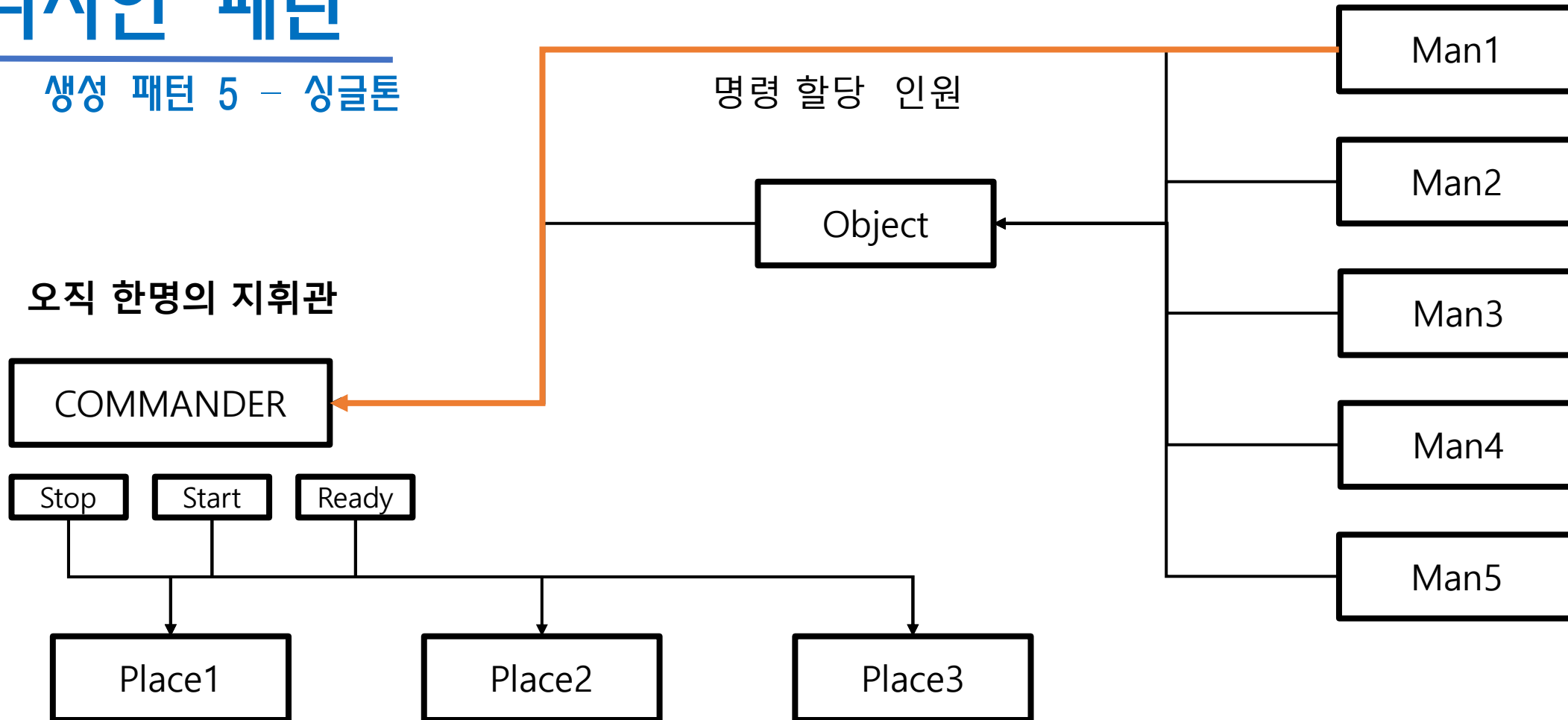
    COMMANDER->SelectObject(human1);
    COMMANDER->Start();
    COMMANDER->Stop();
    COMMANDER->Ready();
}
```

매크로를 전역으로 사용하면
다른 외부 파일에서도
다음 내역과 같이 사용할 수 있다.

디자인 패턴

생성 패턴 5 - 싱글톤

[목록으로](#)



모든 클래스 장소에서
연결된 인원에게 명령을 할 수 있다

디자인 패턴

[목록으로](#)

생성 패턴 최종 정리

※ 디자인 패턴은 사용 목적에 맞게 복합적으로 사용된다.

생성 패턴 [Creational Patten]	인스턴스의 생성과 관리, 삭제 등을 보다 효율적으로 사용하기 위해 사용되는 패턴들의 집합	구성 영역
추상 팩토리 [Abstract Factory]	『팩토리 메소드로 부터 구성된 인스턴스를 관리 할 수 있는 인터페이스』를 새로운 인터페이스로 구성하여 관리하는 패턴	클래스 단위
팩토리 메서드 [Factory Method]	클래스의 필드와 메서드를 인터페이스(<팩토리 로 구성된>명령어 사전)로 부터 관리할 수 있도록 도와주는 패턴	객체 단위
프로토 타입 [Prototype]	같은 내용을 담고 있는 클래스를 여러 개로 복사하여 사용하기 편하게 사용되는 패턴	객체 단위 (Clone 생성)
빌더 [Builder]	『팩토리 메소드로 부터 구성된 인스턴스를 관리 할 수 있는 인터페이스』를 이용하여 새로운 인스턴스를 구성하여 관리하는 패턴	객체 단위
싱글톤 [Singleton]	모든 영역에서 사용되는 기능을 사용하기 위해 전역으로 사용되는 하나의 인스턴스를 구성하는 패턴	객체 단위

구조 패턴 개요

구조 패턴 (Structural Pattern)

더 큰 구조를 형성하기 위해 클래스와 객체를 『합성』 하는 것에 중점을 둔 패턴

- 클래스 구조 패턴 : 상속 기법을 이용하여 인터페이스 및 구현 내용들을 복합
- 객체 구조 패턴 : (인터페이스 및 구현이 아닌), 기능 실현을 위한 객체 합성

특징 ※ 상속을 어떻게 사용하여 구조를 최적화 할 것인가

- 이미 만들어진 클래스를 현재 사용하는 프로그램에 이식하는 경우 사용

고려 사항

- 객체 간의 구조가 개별 객체 간의 차이를 두지 않아야 한다.

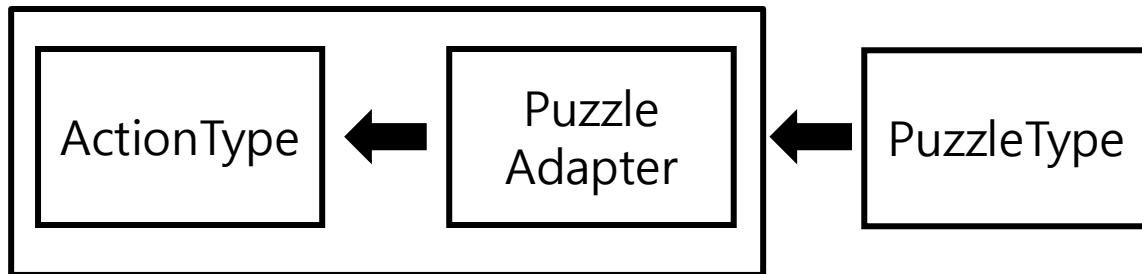
디자인 패턴

[목록으로](#)

구조 패턴 1 - 어댑터

어댑터(Adapter) 패턴

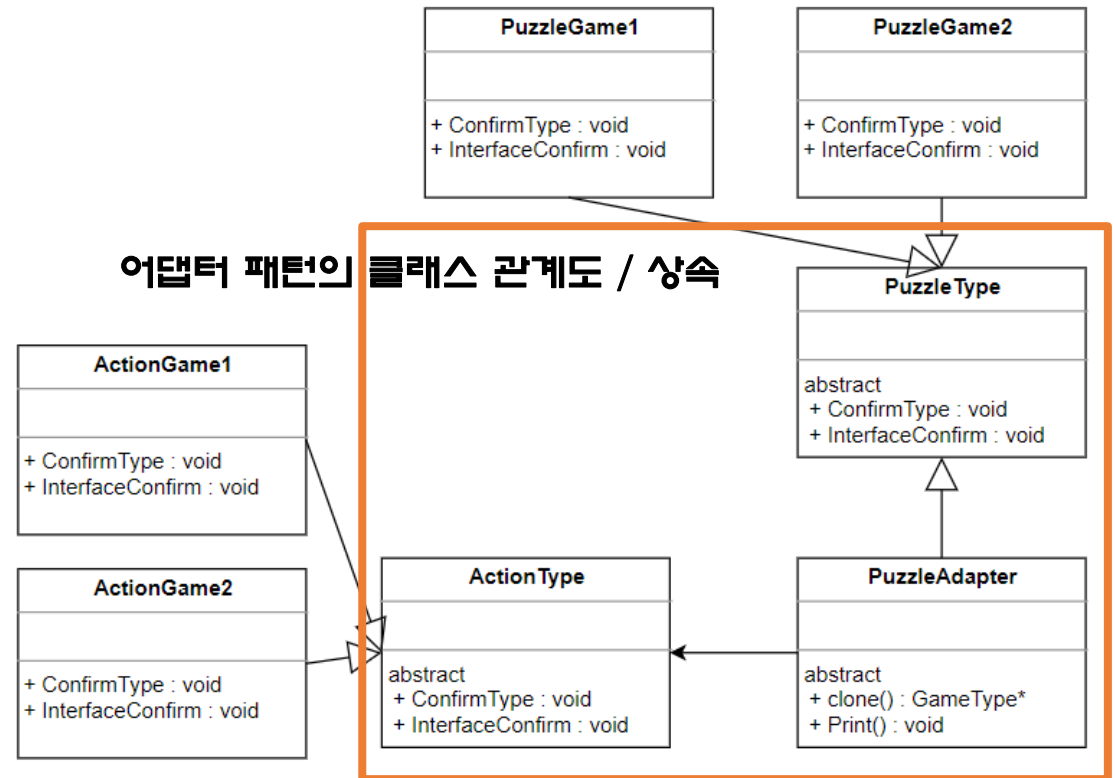
서로 다르게 구현된 인터페이스를
하나의 인터페이스로 구성하는 패턴



2개의 클래스를 하나는 상속, 하나는 연결하는 방식으로 통합하여 하나의 클래스를 만드는 방식

- 일종의 Adapter(어댑터) 역할을 한다.

어댑터 패턴의 클래스 관계도 / 상속



액션 게임의 기능을 가지고 있는 퍼즐게임

디자인 패턴

[목록으로](#)

구조 패턴 1 - 어댑터

PuzzleAdapter

2개의 클래스를 통합한다.

연결

상속

```
class PuzzleType {
private:
public:
    // abstract
    virtual void ConfirmType() = 0;
    virtual void InterfaceConfirm() = 0;
};

class PuzzleStyle1 : public PuzzleType{
    void ConfirmType() { ... }

    void InterfaceConfirm() { ... }
};

class PuzzleStyle2 : public PuzzleType{
    void ConfirmType() { ... }

    void InterfaceConfirm() { ... }
};
```

```
#include "PuzzleType.h"
#include "ActionType.h"

class PuzzleAdapter : public ActionType{
private:
    PuzzleType* puzzleType;
public:
    PuzzleAdapter(PuzzleType* _puzzleType) {
        puzzleType = _puzzleType;
    }
public:
    void ConfirmType() { ... }
    void InterfaceConfirm() { ... }

    void PuzzleConfirmType()
    {
        puzzleType->ConfirmType();
    }
    void PuzzleInterfaceConfirm()
    {
        puzzleType->InterfaceConfirm();
    }
};
```

```
class ActionType
{
private:
public:
    virtual void ConfirmType() = 0;
    virtual void InterfaceConfirm() = 0;
};

class ActionGame1 : public ActionType {
    void ConfirmType() { ... }

    void InterfaceConfirm() { ... }
};

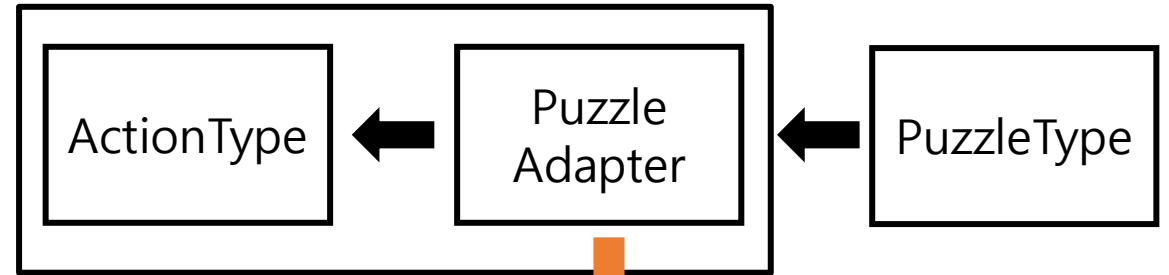
class ActionGame2 : public ActionType {
    void ConfirmType() { ... }

    void InterfaceConfirm() { ... }
};
```


디자인 패턴

[목록으로](#)

구조 패턴 1 - 어댑터



```
#include "PuzzleType.h"
#include "PuzzleAdapter.h"
```

```
int main()
```

```
{
```

```
PuzzleType* puzzleGame1 = new PuzzleStyle1();
PuzzleAdapter* adapter = new PuzzleAdapter(puzzleGame1);
```

```
adapter->ConfirmType();
adapter->InterfaceConfirm();
adapter->PuzzleConfirmType();
adapter->PuzzleInterfaceConfirm();
```



일종의 콘센트를 연결하는 과정과 비슷하다



ActionGame 역할을 담당한다



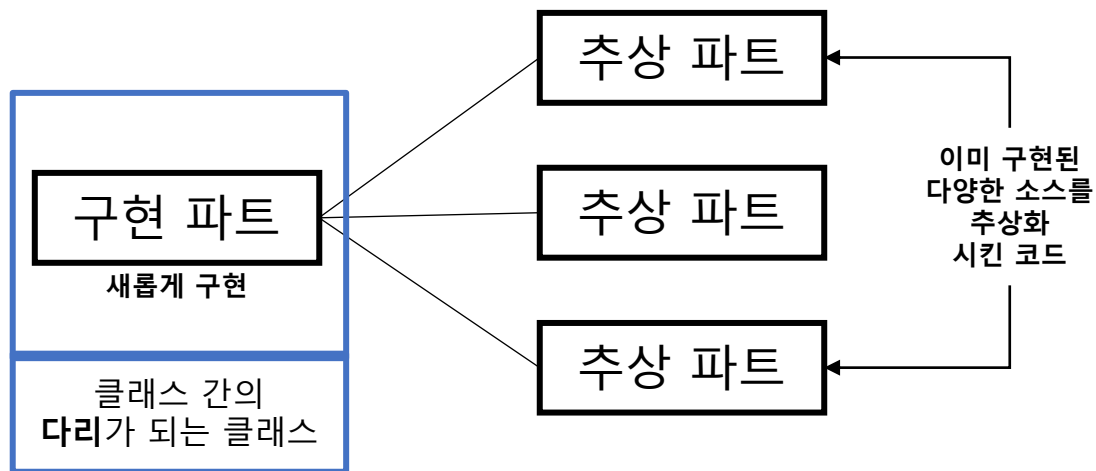
PuzzleGame 역할을 담당한다

구조 패턴 2 - 브릿지

브릿지 패턴(Bridge Patten)

구현 파트와 추상 파트를 서로 독립적으로 분리하는 패턴

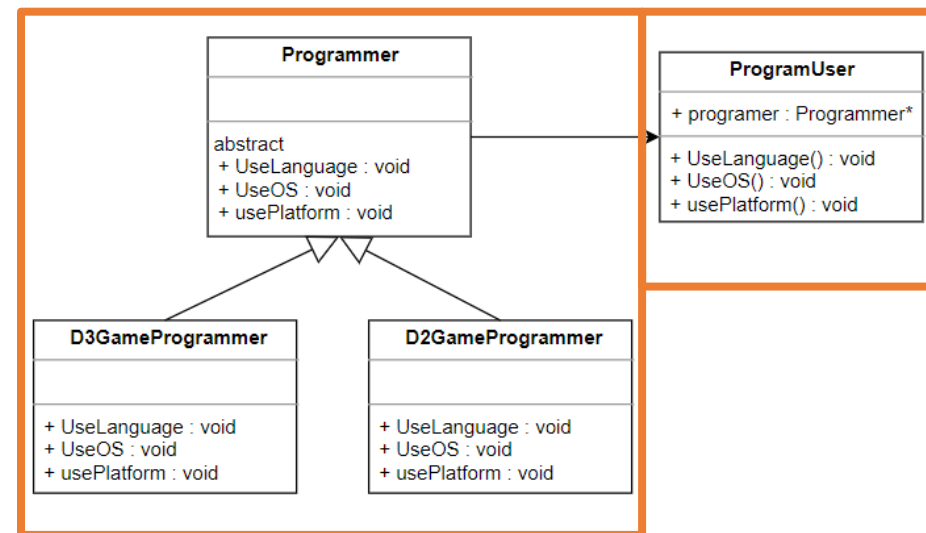
- 추상 파트 : 기능에 대한 내역을 정의 [abstract]
- 구현 파트 : 기능에 대한 실행 내역을 정의



독립적으로 분리, 추상 파트들을 묶거나 해제 시킬 수 있다.

추상 파트

구현 파트



구현에 관련된 클래스가 추상화 된 클래스를 포함한다.

<사용자가 자신의 프로그래머 능력을 확인하는 프로그램>

※ 생성 패턴의 팩토리 메서드와 형태가 비슷한 경우가 있다

브릿지 패턴 : 구조를 매끄럽게 잡기 위한 패턴

팩토리 패턴 : 생성을 매끄럽게 하기 위한 패턴

목적 여부에 맞춰 사용

디자인 패턴

목록으로

구조 패턴 2 – 브릿지

```
class D2GameProgrammer : public Programmer
{
public:
    void UseLanguage() { ... }
    void UseOS() { ... }
    void UsePlatform() { ... }
};

class D3GameProgrammer : public Programmer
{
public:
    void UseLanguage() { ... }
    void UseOS() { ... }
    void UsePlatform() { ... }
};
```

```
class Programmer
{
public:
    // Abstract
    virtual void UseLanguage() = 0;
    virtual void UseOS() = 0;
    virtual void UsePlatform() = 0;
};
```

```
#include "Common.h"
#include "Programmer.h"

class ProgramUser
{
private:
    Programmer* programmer;
public:
    ProgramUser(Programmer* _programmer) { ... }
    void UseLanguage() { ... }
    void UseOS() { ... }
    void UsePlatform() { ... }
};
```

이미 구성되어 있는 Programmer 클래스

다리 역할을 맞는 클래스

디자인 패턴

[목록으로](#)

구조 패턴 2 – 브릿지

```
int main()
{
    D3GameProgrammer* programmer = new D3GameProgrammer;

    ProgramUser* myUser = new ProgramUser(programmer);

    myUser->UseLanguage();
    myUser->UseOS();
    myUser->UsePlatform();

    return 0;
}
```

프로그래머 성향에 대한 클래스

『myUser』 다리 역할을 맡은 클래스

사용 방법

디자인 패턴

구조 패턴 3 - 컴포지트

컴포지트 패턴(Composite Patten)

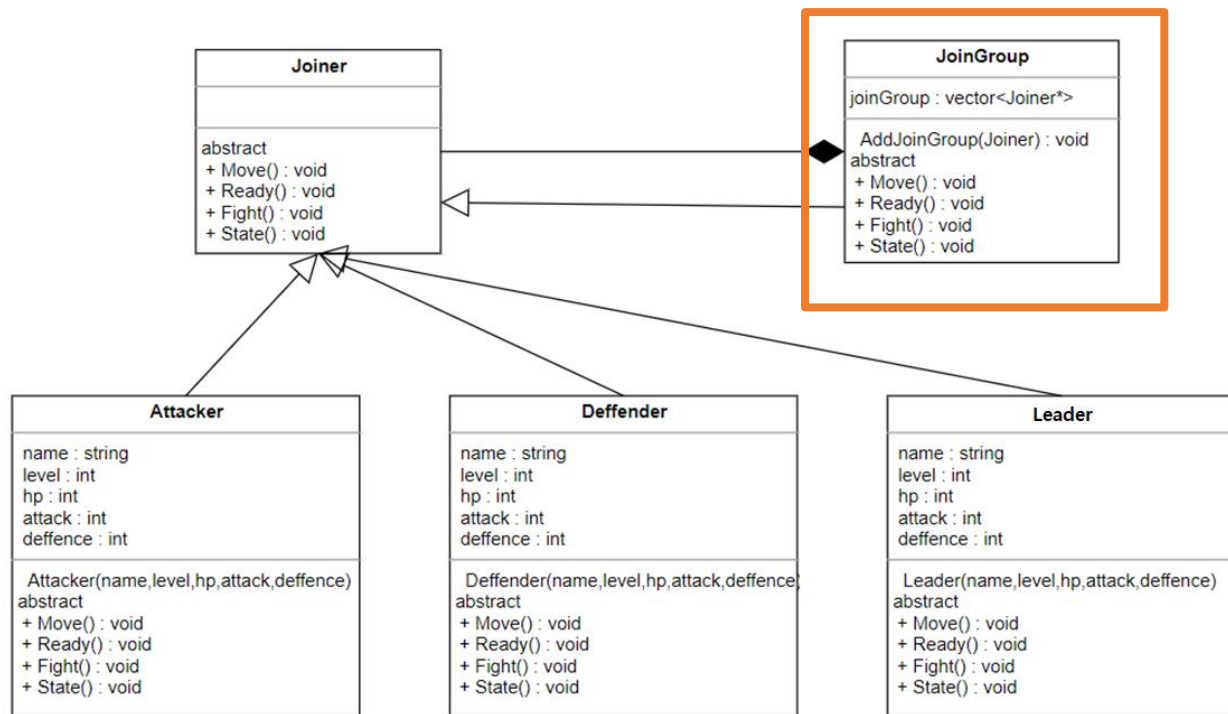
객체와 객체의 그룹을 하나의 인터페이스로
사용 가능하게 하는 패턴

- 객체와 구성을 하나의 인터페이스로 구성한다.
- 하나의 인터페이스로 묶여진 그룹을 함께 관리하는 것이 가능하다.

Ex) 시뮬레이션 게임의 부대지정

목적으로

그룹을 묶어주는 인터페이스



〈가입자들에게 명령 내리는 프로그램〉

디자인 패턴

구조 패턴 3 – 컴포지트

[목록으로](#)

가입 자격이 있는 클래스 [Joiner로 부터 상속받는 클래스]

```
class Attacker : public Joiner
{
private:
    string name;
    int level;
    int hp;
    int attack;
    int deffence;

public:
    // override 내역
    void Move() { ... }
    void Ready() { ... }
    void Fight() { ... }
    void State() { ... }

    // Getter Setter
    string GetData() { return name; }
    int GetLevel() { return level; }
    int GetHP() { return hp; }
    int GetAttack() { return attack; }
    int GetDeffence() { return deffence; }

    // 생성자
    Attacker(string _name, int _level, int _hp,
        int _attack, int _deffence) { ... }
};
```

```
class Deffender : public Joiner
{
private:
    string name;
    int level;
    int hp;
    int attack;
    int deffence;

public:
    // override 내역
    void Move() { ... }
    void Ready() { ... }
    void Fight() { ... }
    void State() { ... }

    // Getter Setter
    string GetData() { return name; }
    int GetLevel() { return level; }
    int GetHP() { return hp; }
    int GetAttack() { return attack; }
    int GetDeffence() { return deffence; }

    // 생성자
    Deffender(string _name, int _level, int _hp,
        int _attack, int _deffence) { ... }
};
```

```
class Leader : public Joiner
{
private:
    string name;
    int level;
    int hp;
    int attack;
    int deffence;

public:
    // override 내역
    void Move() { ... }
    void Ready() { ... }
    void Fight() { ... }
    void State() { ... }

    // Getter Setter
    string GetData() { return name; }
    int GetLevel() { return level; }
    int GetHP() { return hp; }
    int GetAttack() { return attack; }
    int GetDeffence() { return deffence; }

    // 생성자
    Leader(string _name, int _level, int _hp,
        int _attack, int _deffence) { ... }
};
```

디자인 패턴

구조 패턴 3 – 컴포지드

목록으로

```
class Joiner
{
public:
    //abstract
    virtual void Move() = 0;
    virtual void Ready() = 0;
    virtual void Fight() = 0;
    virtual void State() = 0;
};
```

가입을 했을 시 받을 수 있는
명령 메서드[모음집]



```
class JoinGroup : public Joiner
{
private:
    vector<Joiner*> joinGroup;
public:

    void Move() {
        if (joinGroup.empty()){ return;}
        for (int i = 0; i < joinGroup.size(); i++)
        {
            joinGroup.at(i)->Move();
        }
    };
    void Ready() { ... }
    void Fight() { ... }
    void State() { ... }

    void AddJoinGroup(Joiner* joiner)
    {
        if (joinGroup.size() < joinGroup.capacity())
        {
            joinGroup.push_back(joiner);
        }
    }
    JoinGroup() { ... }
};
```

관리 모델로 Vector
사용

그룹에 가입된 클래스
전체에게 명령을 하달한다

그룹에 클래스를
가입시킨다.

디자인 패턴

구조 패턴 3 - 컴포지드

[목록으로](#)

Main 호출 예시) 사용방법

```
Attacker* attacker1 = new Attacker("공격자1", 10, 10, 10, 10);  
Attacker* attacker2 = new Attacker("공격자2", 20, 20, 20, 20);  
Deffender* deffence1 = new Deffender("방어자1", 10, 10, 10, 10);  
Deffender* deffence2 = new Deffender("방어자2", 20, 20, 20, 20);  
Leader* leader1 = new Leader("리더1", 10, 10, 10, 10);  
Leader* leader2 = new Leader("리더2", 20, 20, 20, 20);
```

```
JoinGroup* joinGroup = new JoinGroup();
```

```
joinGroup->AddJoinGroup(attacker1);  
joinGroup->AddJoinGroup(attacker2);  
joinGroup->AddJoinGroup(deffence1);  
joinGroup->AddJoinGroup(deffence2);  
joinGroup->AddJoinGroup(leader1);  
joinGroup->AddJoinGroup(leader2);
```

```
joinGroup->State();  
joinGroup->Fight();  
joinGroup->Ready();  
joinGroup->Move();
```

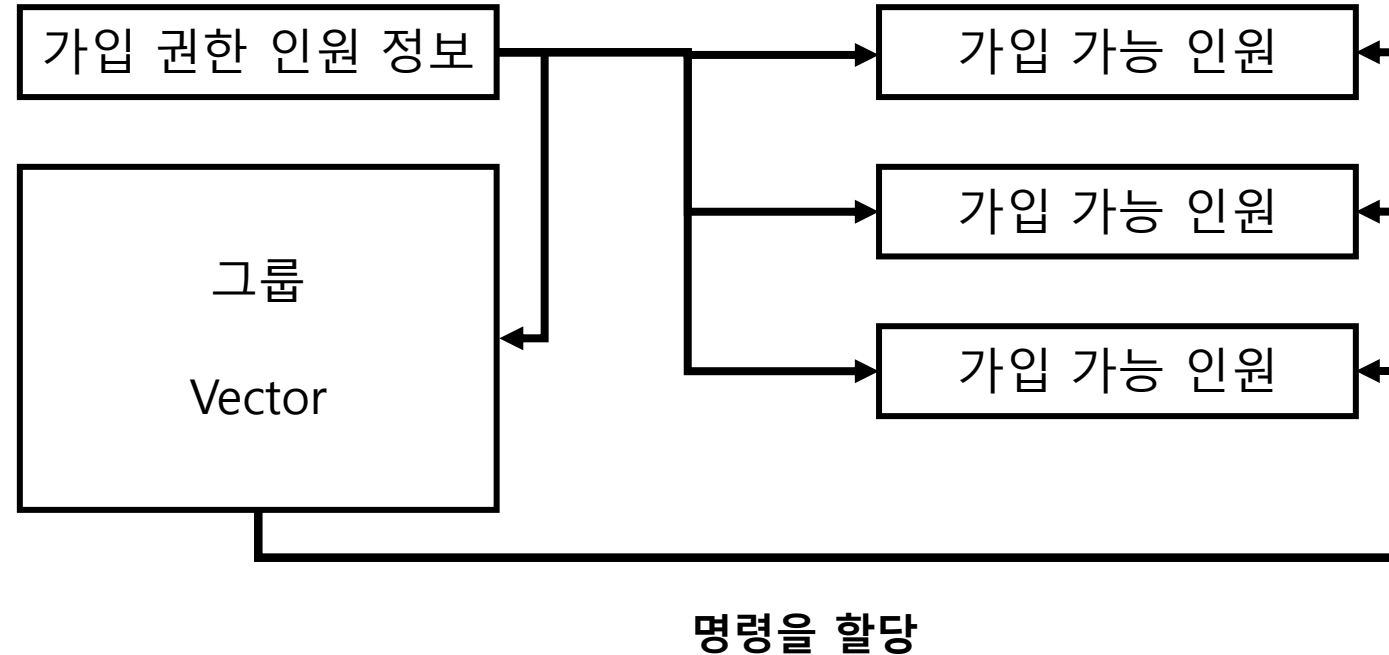
가입 권한이 있는 클래스를 생성한다.

저장 그룹을 생성한다

그룹에 클래스들을 저장시킨다.

그룹 전체에게 명령을 내린다

구조 패턴 3 – 컴포지드



1. 그룹은 가입 권한이 있는 인원을 확인.
2. 가입 가능 인원을 가입시킨다.
3. 가입한 인원을 단체 관리 가능하다

디자인 패턴

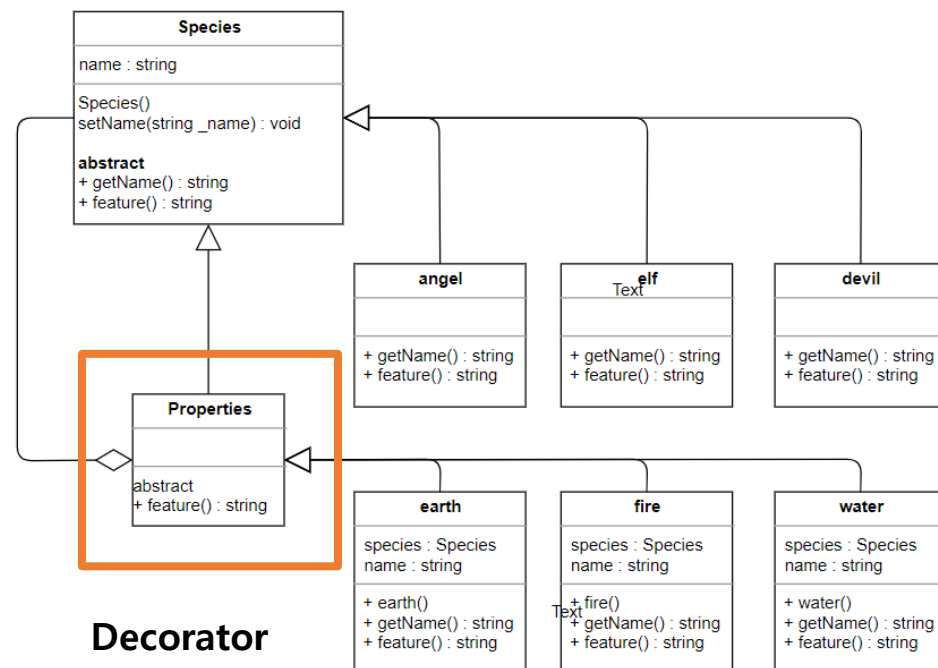
[목록으로](#)

구조 패턴 4 - 데코레이터 패턴

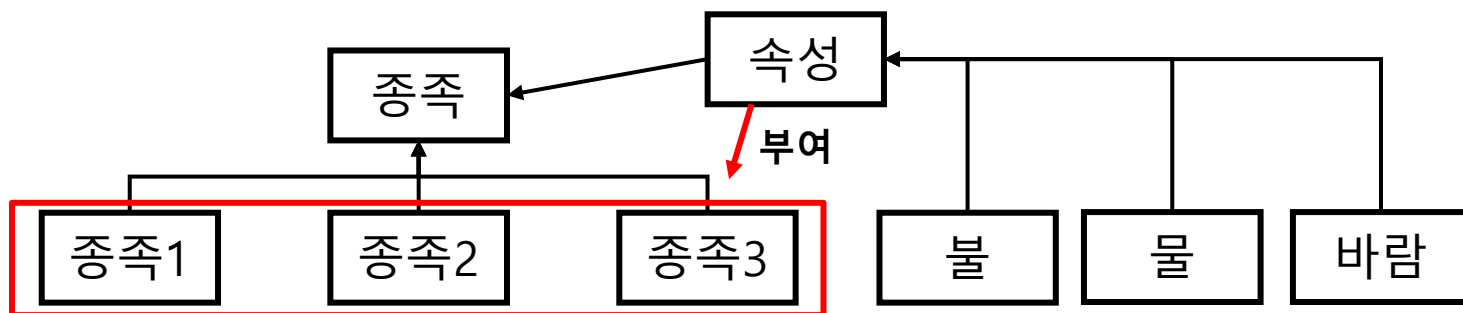
데코레이터(Decorator)

객체의 결합을 하는 방식으로, 개체가 가지는 기능을 동적으로 확장하는 패턴

- 추가 기능에 대한 객체를 포괄적으로 삽입, 삭제가 가능하다.
- 객체로부터 파생되는 다양한 조합들을 동적으로 구현이 가능하다.



<종족 생성 이후 그 종족에게 속성을 부여하기 위한 프로그램>



확장 방법

```
Species* species = new angel;
species = new earth(species);
```

디자인 패턴

[목록으로](#)

구조 패턴 4 – 데코레이터 패턴

```
class Species
{
private:
    string name;
public:
    void SetName(string _name) { name = _name; }
    virtual string getName() { return name; }
    virtual string feature() = 0;

    Species() { name = "\\0"; }
};
```

```
class Properties : public Species
{
public:
    virtual string feature() = 0;
};
```

추가해야 할 속성에 대한 객체

기존 종족에 대한 설정 요소

```
class angel : public Species
{
public:
    angel() { SetName("천사"); }
    string feature() {
        return "하얀 날개를 가지고 있다.";
    }
};
```

```
class elf : public Species
{
public:
    elf() { SetName("엘프"); }
    string feature() {
        return "긴 귀를 가지고 있다.";
    }
};
```

```
class devil : public Species
{
public:
    devil() { SetName("악마"); }
    string feature() {
        return "검은 날개를 가지고 있다.";
    }
};
```

디자인 패턴

목록으로

구조 패턴 4 - 데코레이터 패턴

```
class Properties : public Species
{
public:
    virtual string feature() = 0;
};
```

추가되는 속성들



사용되는 예시)

천사 종족 -> 지구 속성 부여

```
Species* species = new angel;
cout << species->getName() << " : " << species->feature() << endl;

species = new earth(species);
cout << species->getName() << " : " << species->feature() << " 속성 " << endl;
```

천사 : 하얀 날개를 가지고 있다.
대지의 천사 : 하얀 날개를 가지고 있다. + 지구 속성

```
class fire : public Properties
{
    Species* species;
    string name;
public:
    fire(Species* _species) {
        this->species = _species;
        name = "화염의 " + species->getName();
    };
    string getName() { return name; }
    string feature() { return species->feature() + " + 화염"; }
```

```
class water : public Properties
{
    Species* species;
    string name;
public:
    water(Species* _species) {
        this->species = _species;
        name = "수신의 " + species->getName();
    }
    string getName() { return name; }
    string feature() { return species->feature() + " + 물"; }
```

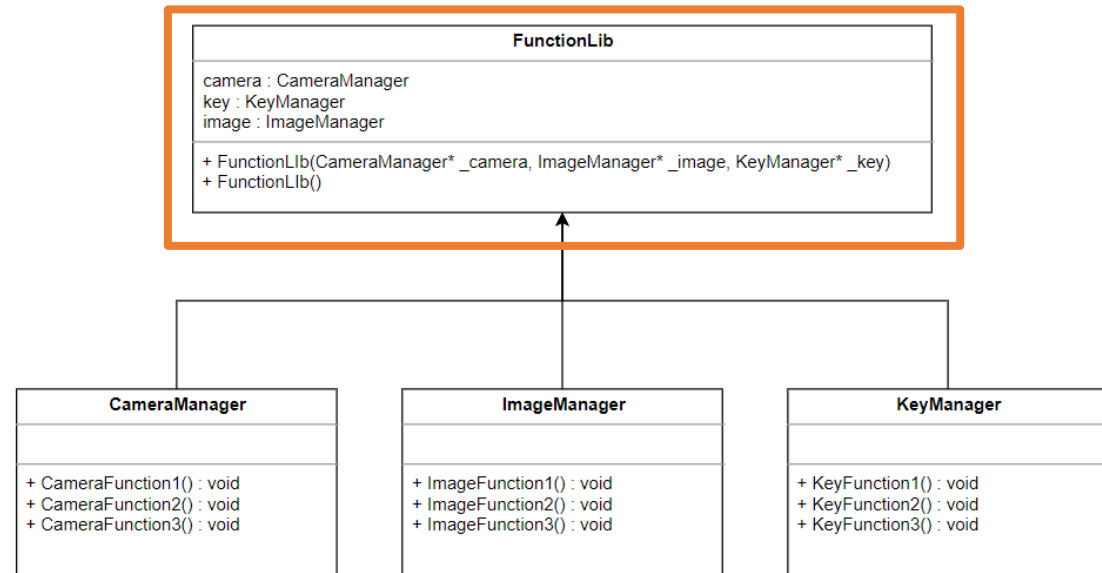
```
class earth : public Properties
{
    Species* species;
    string name;
public:
    earth(Species* _species) {
        this->species = _species;
        name = "대지의 " + species->getName();
    }
    string getName() { return name; }
    string feature() { return species->feature() + " + 지구"; }
```

구조 패턴 5 – 퍼사드

퍼사드(Facade)

복잡한 서브 클래스들의 인터페이스를 하나로 묶어
사용이 가능하도록 한 패턴

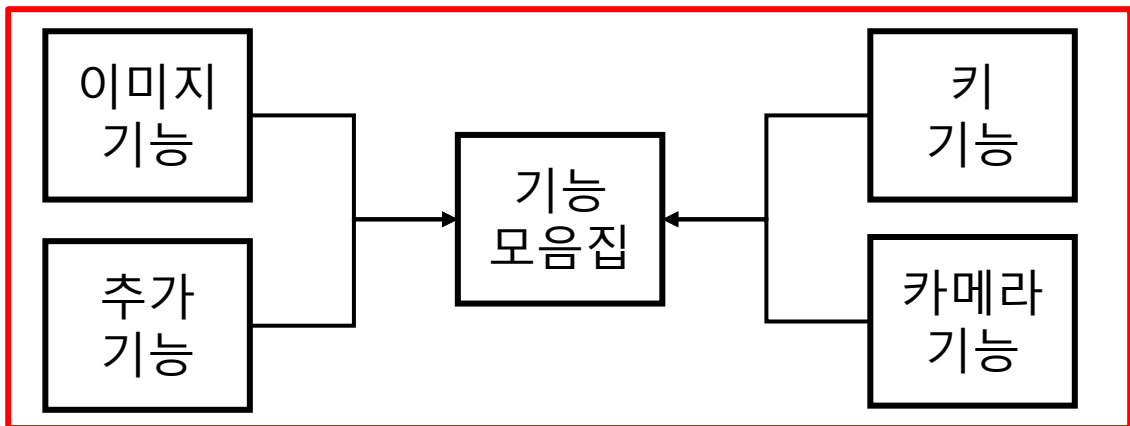
- 일종의 사전 시스템을 만들어, 필요한 기능에 대한 인터페이스를 필요할 때에만 사용한다.
- 클래스들의 외관 형태, 내부를 알지는 못하지만 그 기능을 사용할 수 있는 패턴을 의미한다.



<게임 제작에 필요한 기능들을 모아 놓은 사전>



PS. 싱글 톤 베이스와 연동하여 사용하면 유용한 패턴이 될 것 같습니다.



디자인 패턴

구조 패턴 5 - 퍼사드

목록으로

```
class CameraManager
{
public:
    void CameraFunction1() { cout << "카메라 기능1"; }
    void CameraFunction2() { cout << "카메라 기능2"; }
    void CameraFunction3() { cout << "카메라 기능3"; }
};
```

```
class ImageManager
{
public:
    void ImageFunction1() { cout << "이미지 기능1"; }
    void ImageFunction2() { cout << "이미지 기능2"; }
    void ImageFunction3() { cout << "이미지 기능3"; }
};
```

```
class KeyManager
{
public:
    void KeyFunction1() { cout << "키 입력 함수"; }
    void KeyFunction2() { cout << "키 기능 함수"; }
    void KeyFunction3() { cout << "키 출력 함수"; }
};
```

각각의 함수가
그 기능을 작동
하도록 따로
구현

Main

메인에서 일일이 다 불러줘야 하며,
변수명도 제각각 다르다

```
// 추가적으로 사용할 기능 모음집
CameraManager* camera = new CameraManager;
ImageManager* image = new ImageManager;
KeyManager* key = new KeyManager;
```

디자인 패턴

목록으로

구조 패턴 5 - 퍼사드

객체를
연동시켜준다.

```
class CameraManager
{
public:
    void CameraFunction1() { cout << "카메라 기능1"; }
    void CameraFunction2() { cout << "카메라 기능2"; }
    void CameraFunction3() { cout << "카메라 기능3"; }
};
```

```
class ImageManager
{
public:
    void ImageFunction1() { cout << "이미지 기능1"; }
    void ImageFunction2() { cout << "이미지 기능2"; }
    void ImageFunction3() { cout << "이미지 기능3"; }
};
```

```
class KeyManager
{
public:
    void KeyFunction1() { cout << "키 입력 함수"; }
    void KeyFunction2() { cout << "키 기능 함수"; }
    void KeyFunction3() { cout << "키 출력 함수"; }
};
```



```
class FunctionLib
{
public:
    CameraManager* camera;
    ImageManager* image;
    KeyManager* key;
```

Type1 외부에서 기능 부여

```
FunctionLib(
    CameraManager* _camera,
    ImageManager* _image,
    KeyManager* _key)
```

```
{
    camera = _camera;
    image = _image;
    key = _key;
}
```

Type2 내부에서 기능 부여

```
FunctionLib()
{
    camera = new CameraManager;
    image = new ImageManager;
    key = new KeyManager;
}
```

```
};
```

디자인 패턴

[목록으로](#)

구조 패턴 5 – 퍼사드

```
// type1 밖에서 사전을 넣어주는 방법

// 추가적으로 사용할 기능 모음집
CameraManager* camera = new CameraManager;
ImageManager* image = new ImageManager;
KeyManager* key = new KeyManager;

// 기능 사전에 넣기
FunctionLib* functionLib = new FunctionLib(camera, image, key);
functionLib->camera->CameraFunction1();

// type2
// Facade 클래스 내부에 직접적으로 사전을 직접 작성하는 방법
FunctionLib* functionLib2 = new FunctionLib();
functionLib2->camera->CameraFunction1();
```

기능들을 라이브러리로 묶어
그에 필요한 기능들을 사용할 수
있다.

Type2
Façade 패턴이 유용해 지는 예시

디자인 패턴

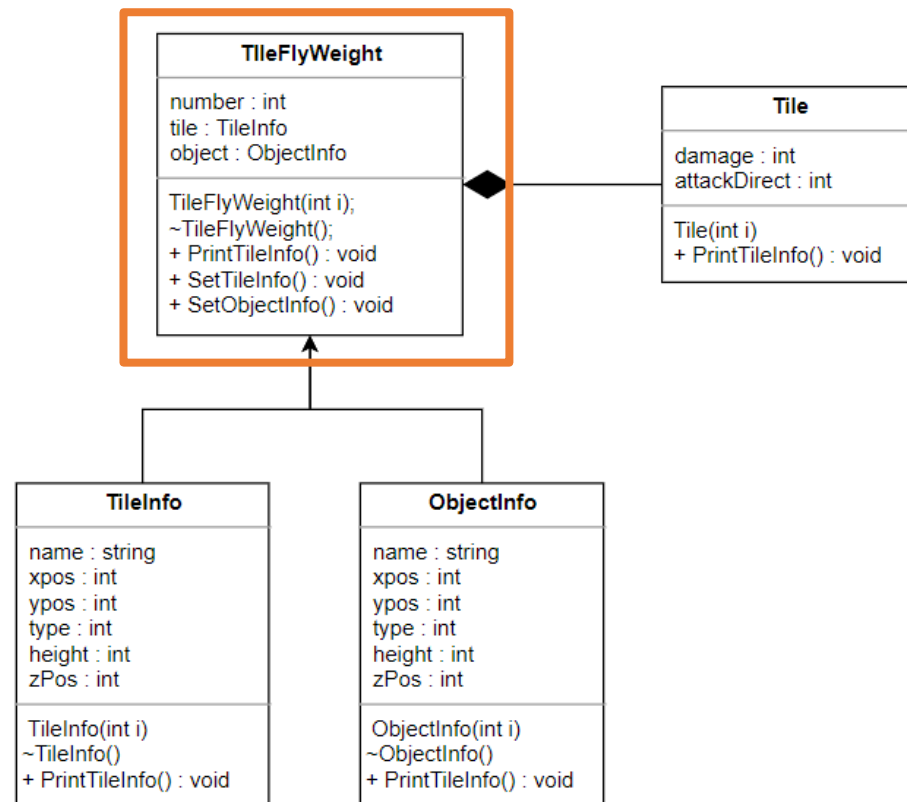
[목록으로](#)

구조 패턴 6 – 플라이웨이트

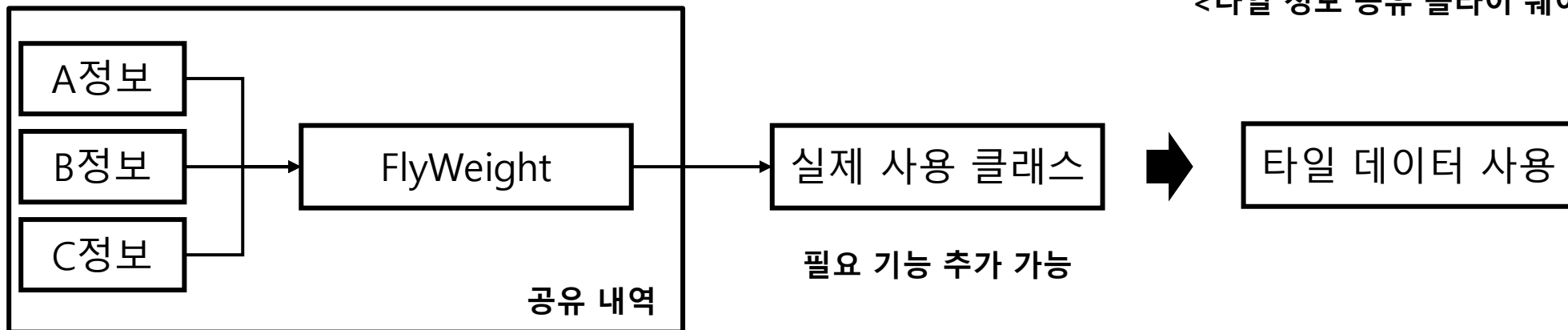
플라이웨이트(Flyweight)

객체가 있으면 공용해서 사용하고, 없으면 새로 생성하여 관리하는 패턴

- 데이터를 공유하여 메모리를 절약할 수 있는 패턴
- 공통으로 사용되는 객체는 새로 생성해서 사용하지 않고 공유를 통해 효율적으로 자원을 활용



<타일 정보 공유 플라이 웨이트 프로그램>



디자인 패턴

[목록으로](#)

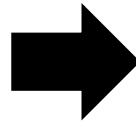
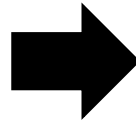
구조 패턴 6 – 플라이웨이트

```
class TileInfo
{
    string name;    // 타일 이름
    int xpos;      // X좌표
    int ypos;      // Y좌표
    int type;      // 타일 속성
    int height;    // 높이
    int zPos;      // Z-order

public:
    void PrintTileInfo();
    TileInfo(int i);
    ~TileInfo();
};
```

```
class ObjectInfo
{
    string name;
    int xpos;      // X좌표
    int ypos;      // Y좌표
    int type;      // 타일 속성
    int height;    // 높이
    int zPos;      // Z-order

public:
    void PrintObjectInfo();
    ObjectInfo(int i);
    ~ObjectInfo();
};
```

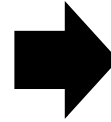


```
class TileInfo;
class ObjectInfo;

class TileFlyWeight
{
    int number;
    TileInfo* tile;
    ObjectInfo* object;

public:
    TileFlyWeight(int i);
    ~TileFlyWeight();
    void PrintTileInfo();
    void SetTileInfo();
    void SetObjectInfo();
};
```

공유 내역



```
class Tile : public TileFlyWeight
{
    int damage;
    int attackDirect;

public:
    Tile(int i) : damage(0), attackDirect(0), TileFlyWeight(i) {};
    void PrintTileInfo();
};
```

추가 내역

```
void Tile::PrintTileInfo()
{
    TileFlyWeight::PrintTileInfo();
    // 추가
    cout << "————— 추가 정보 —————" << endl;
    cout << "damage : " << damage << endl;
    cout << "attackDirect : " << attackDirect << endl;
}
```

타일 정보 가져와 사용하는 클래스

디자인 패턴

구조 패턴 6 – 플라이웨이트

```
void TileFlyWeight::PrintTileInfo()
{
    if (tile)tile->PrintTileInfo();
    if (object)object->PrintObjectInfo();
}

void TileFlyWeight::SetTileInfo()
{
    tile = new TileInfo(number);
}

void TileFlyWeight::SetObjectInfo()
{
    object = new ObjectInfo(number);
}
```

공유할 내역을 따로 정의하는
TileFlyWeight

목록으로

```
void ObjectInfo::PrintObjectInfo()
{
    cout << "————— 오브젝트 정보 —————" << endl;
    cout << "이름 : " << name << endl;
    cout << "xpos : " << xpos << endl;
    cout << "ypos : " << ypos << endl;
    cout << "type : " << type << endl;
    cout << "height : " << height << endl;
    cout << "zPos : " << zPos << endl;
    cout << "—————" << endl;
}
```

오브젝트 정보

```
ObjectInfo::ObjectInfo(int i) :
    name("오브젝트"),
    xpos(0),
    ypos(0),
    type(0),
    height(0),
    zPos(0)
{
    name = name + to_string(i);
}
```

```
void TileInfo::PrintTileInfo()
{
    cout << "————— 타일 정보 —————" << endl;
    cout << "이름 : " << name << endl;
    cout << "xpos : " << xpos << endl;
    cout << "ypos : " << ypos << endl;
    cout << "type : " << type << endl;
    cout << "height : " << height << endl;
    cout << "zPos : " << zPos << endl;
    cout << "—————" << endl;
}
```

타일 정보

```
TileInfo::TileInfo(int i) :
    name("타일"),
    xpos(0),
    ypos(0),
    type(0),
    height(0),
    zPos(0)
{
    name = name + to_string(i);
}
```

구조 패턴 6 – 플라이웨이트

```
Tile* tile[SIZEX][SIZEY];  
for (int i = 0; i < SIZEX * SIZEY; i++)  
{  
    tile[i / SIZEX][i % SIZEY] = new Tile(i);  
    tile[i / SIZEX][i % SIZEY]->SetTitleInfo();  
}
```

```
//  
tile[0][1]->SetObjectInfo();  
tile[0][1]->PrintTileInfo();
```

```
//  
for (int i = 0; i < SIZEX * SIZEY; i++)  
{  
    delete tile[i / SIZEX][i % SIZEY];  
    tile[i / SIZEX][i % SIZEY] = NULL;  
}
```

타일 동적 할당

타일 세팅

할당해제

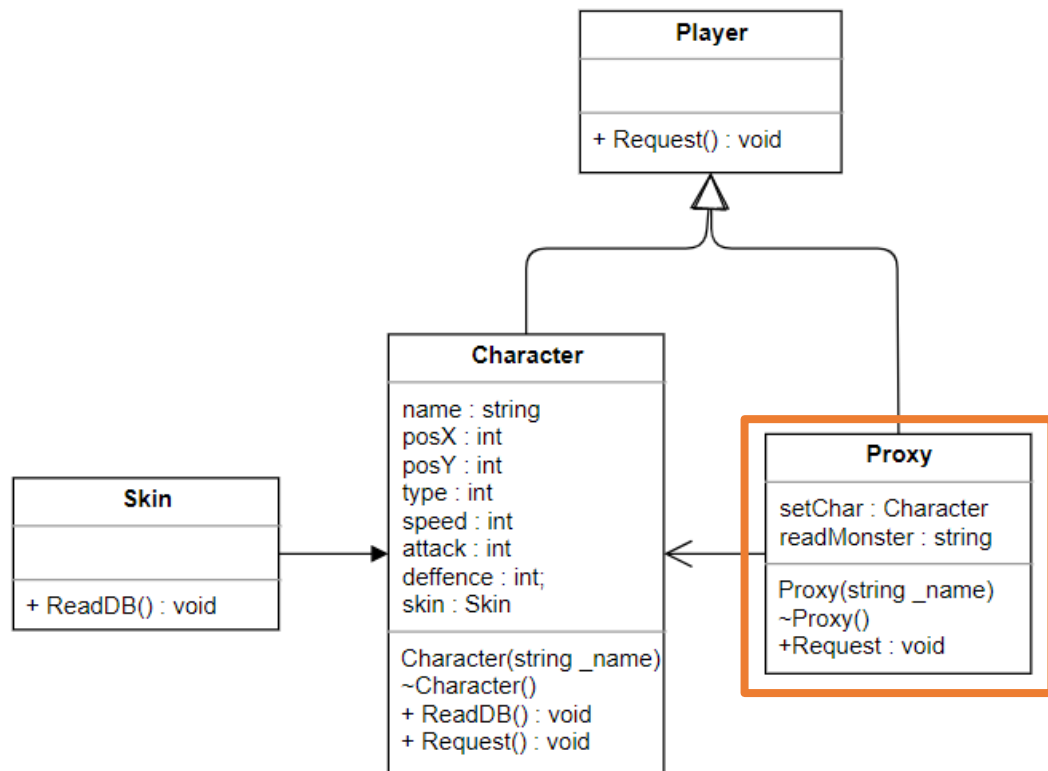
덴글리 포인터 : 할당되지 않은 공간(해제된)을 가리키는 포인터
NULL로 지정하여 해제한다.

구조 패턴 7 - 프록시

프록시(Proxy)

대리자로서 일을 맡기면 그 일을 처리하고 완료되면 그 결과를 알려주는 패턴

- 실제 기능을 수행하는 객체를 가상의 객체를 사용해 로직의 흐름을 제어하는 디자인 패턴
- 목적에 따라 프록시의 종류가 다양해 진다
 - 가상 프록시 : 미리 생성하기 힘든 객체 접근
 - 보호 프록시 : 접근 권한 제어
 - 캐싱 프록시 : 캐시 메모리 기능 대체
 - 원격 , 방화벽, 스마트 레퍼런스, 동기화,
 - 복잡도, 지연 복사 등등



<캐릭터에 대한 정보를 불러오는 프로그램>

디자인 패턴

구조 패턴 7 - 프록시

[목록으로](#)

```
class Player
{
public:
    virtual void Request() = 0;
};
```

```
class Character : public Player
{
    string name;
    int posX;
    int posY;
    int type;
    int speed;
    int attack;
    int deffence;
    Skin* skin;

    void ReadDB() {};

public:
    void Request()
    {
        cout << name << " 캐릭터를 불러왔습니다" << endl;
    };
    Character(string _name) { name = _name; };
    ~Character() {};
};
```

```
class Proxy : public Player
{
    Character* setChar;
    string readMonster;

public:
    void Request()
    {
        if (!setChar) setChar = new Character(readMonster);
        setChar->Request();
    }

    Proxy(string _name) : readMonster(_name), setChar(NULL) {};
    ~Proxy() { if (setChar) delete setChar; }
};
```

<프록시를 통해 Character의 기능을 불러오는 중...>

구조 패턴 7 – 프록시

```
int main()
{
    Player* playerChar = new Proxy("헤카림");

    playerChar->Request();

    delete playerChar;
}
```

<DB나 기타 기능 등을 다 불러오면 Request요청이 완료된다.>

헤카림 캐릭터를 불러왔습니다

대표적인 예시) 스마트 포인터

※ 스마트 포인터

- 포인터로 사용된 변수의 메모리를 자동으로 해제시켜주는 포인터



어떻게?

Unique_ptr	실제 데이터에 대한 소유권을 오직 하나만 가지도록 하는 포인터
Shared_ptr	실제 데이터에 대한 소유권을 여러 개의 스마트 포인터가 공유하면서 참조 카운트를 작성 참조 카운트가 0일 때 메모리를 해제하는 포인터 (reset 사용 으로 감소)
Weak_ptr	Shared_ptr에서 참조 카운트가 없는 형태

디자인 패턴

[목록으로](#)

구조 패턴 최종 정리

※ 디자인 패턴은 사용 목적에 맞게 복합적으로 사용된다.

↓ : 패턴 제한 이유

↑ : 패턴 생성 목적

구조 패턴 [Structural Patten]	더 큰 구조를 형성하기 위해 클래스와 객체를 합성하는 것에 중점을 둔 패턴	구성 영역
어댑터 [Adapter]	서로 다르게 구현 된 두개의 인터페이스를 하나의 인터페이스로 구성하는 패턴	클래스 단위
브릿지 [Bridge]	구현 파트와 추상 파트를 서로 독립적으로 분리하는 패턴	객체 단위
컴포지트 [Composite]	객체와 객체의 그룹을 하나의 인터페이스로 사용 가능하게 하는 패턴	객체 단위
데코레이터 [Decorator]	객체의 결합을 하는 방식으로, 객체가 가지는 기능을 동적으로 확장하는 패턴	객체 단위
퍼사드 [Fecade]	복잡한 서브 클래스들의 인터페이스를 하나로 묶어 사용이 가능하도록 한 패턴	객체 단위
플라이웨이트 [FlyWeight]	객체가 있으면 공용해서 사용하고, 없으면 새로 생성하여 관리하는 패턴	객체 단위
프록시 [proxy]	대리자로서 일을 맡기면 그 일을 처리하고 완료되면 그 결과를 알려주는 패턴	객체 단위

디자인 패턴

행동 패턴

	디자인 패턴	예제 프로그램	페이지
행 동 패 턴	인터프리터(Interpreter)	클라이언트 / 서버 암호화 전송 예제 프로그램	p. 67
	템플릿메서드(Template Method)	몬스터의 행동을 포괄적으로 관리해 주는 프로그램	p. 70
	책임연쇄(Chain of Responsibility)	퀘스트를 단계별로 관리해 줄 수 있는 Chain 프로그램	p. 73
	명령(Command)	특정 유저의 행동을 관리할 수 있는 프로그램	p. 77
	이터레이터(iterator)	적들의 데이터를 Iterator로 관리하는 프로그램	p. 80
	미디에이터(Mediator)	인벤토리를 관리하는 중재자 역할이 있는 프로그램	p. 83
	메멘토(Memento)	플레이어의 움직임을 기록하고 되감기 하는 프로그램	p. 86
	관찰자(Observer)	접속자 전체 리스트에 공지를 내려주는 운영 프로그램	p. 89
	전략(Strategy)	스킬 관리 프로그램	p. 92
	상태(State)	플레이어의 행동을 관리하는 프로그램	p. 94
	방문자(Visitor)	플레이어의 장비를 세팅하는 프로그램	p. 96

생성 / 구조 패턴	p. 10
------------	-----------------------

참고문헌	p. 102
------	------------------------

행동 패턴

행동 패턴(Behavioral Pattern)

변수와 메소드를 관리했던 생성/구조 패턴과는 달리 알고리즘을 어느 객체에 적용할지에 대한 패턴

- 행동 클래스 패턴 : 클래스 사이의 행동 책임을 분산하기 위해 상속을 사용한 패턴
- 행동 객체 패턴 : 상속 보다는 복합을 통해서 객체 사이에 행동 처리를 분산시키는 패턴

특징

- 객체 간의 제어 구조보다는 객체들을 어떻게 연결시킬 것인지에 중점을 두어야 한다.

디자인 패턴

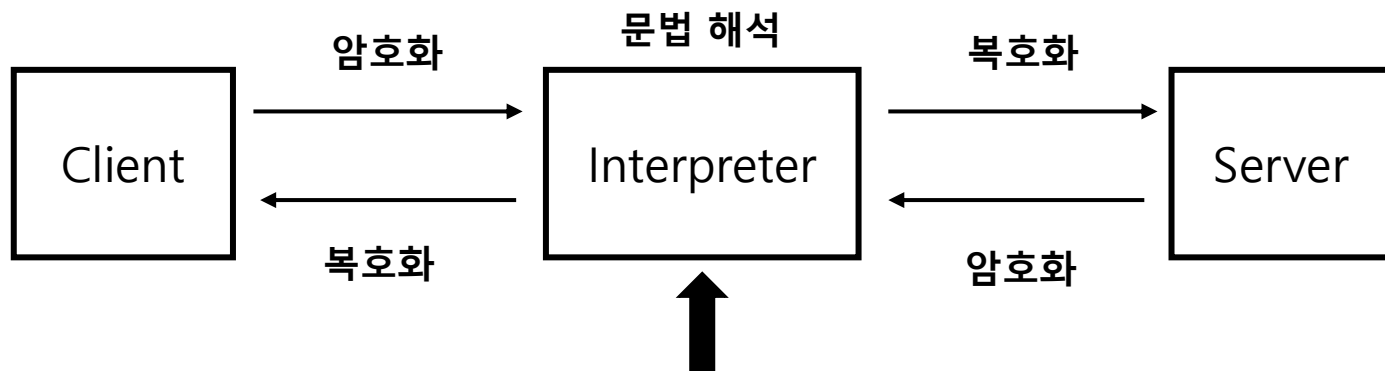
목적으로

행동 패턴 1 - 인터프리터

인터프리터(Interpreter)

문법 규칙을 클래스화, 일련의 규칙으로 정의된 언어 해석 패턴

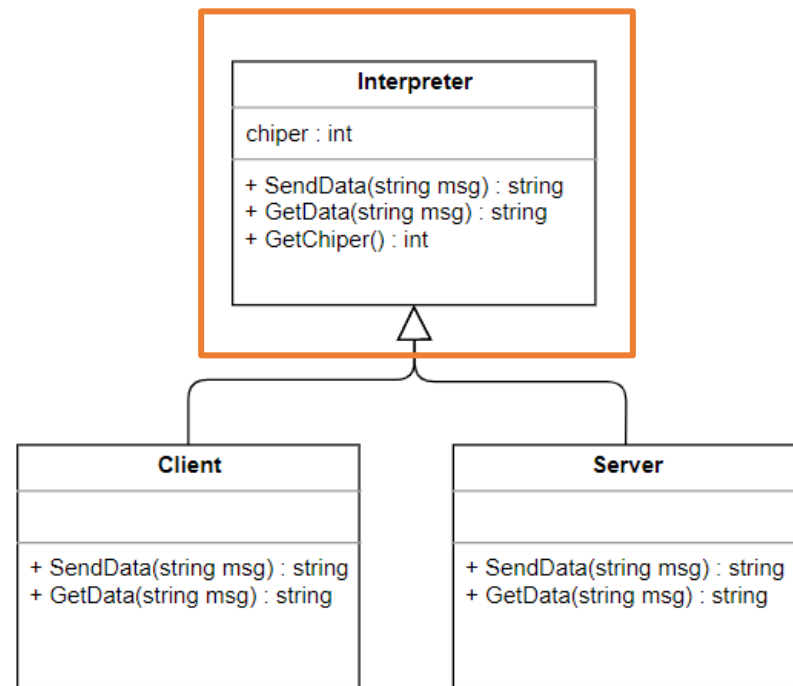
- 클래스 간의 컴파일러(공통된 언어)를 구현하는 것과 같다.
- 예시) 네트워크 통신, 커맨드 해석기, SQL 구문 등



프로토콜과 같은 기능을 담당하기도 한다

HTTP / FTP / SMTP / SSH / TELNET 등

해독기 기능을 하는 인터프리터



<클라이언트 / 서버 암호화 전송 예제 프로그램>

디자인 패턴

목록으로

행동 패턴 1 - 인터프리터

```
class Server : public Interpreter
{
public:
    string GetData(string msg)
    {
        char data[32] = { '\0' };

        for (int i = 0; i < msg.length(); i++)
        {
            data[i] = msg.at(i) - GetChiper();
        }

        string tempMsg(data);
        return tempMsg;
    }

    string SendData(string msg)
    {
        char data[32] = { '\0' };

        for (int i = 0; i < msg.length(); i++)
        {
            data[i] = msg.at(i) + GetChiper();
        }

        string tempMsg(data);
        return tempMsg;
    }
};
```

문법 규칙 구현
전치암호

```
class Interpreter
{
    int chiper = 4;
public:
    virtual string GetData(string msg) = 0;
    virtual string SendData(string msg) = 0;

    int GetChiper() { return chiper; }
};
```

규칙을 작성한 클래스

```
class Client : public Interpreter
{
public:
    string GetData(string msg)
    {
        char data[32] = { '\0' };

        for (int i = 0; i < msg.length(); i++)
        {
            data[i] = msg.at(i) - GetChiper();
        }

        string tempMsg(data);
        return tempMsg;
    }

    string SendData(string msg)
    {
        char data[32] = { '\0' };

        for (int i = 0; i < msg.length(); i++)
        {
            data[i] = msg.at(i) + GetChiper();
        }

        string tempMsg(data);
        return tempMsg;
    }
};
```

디자인 패턴

[목록으로](#)

행동 패턴 1 - 인터프리터

```
string sendData;
```

```
Interpreter* server = new Server();  
sendData = server->SendData("Hello");
```

```
Interpreter* client = new Client();  
cout << client->GetData(sendData) << endl;
```

서버에서 데이터 전송

```
전송할 데이터 : Hello  
저장하는 데이터 : Lipps  
클라이언트가 받은 데이터 : Hello
```

클라이언트에서 받은 데이터

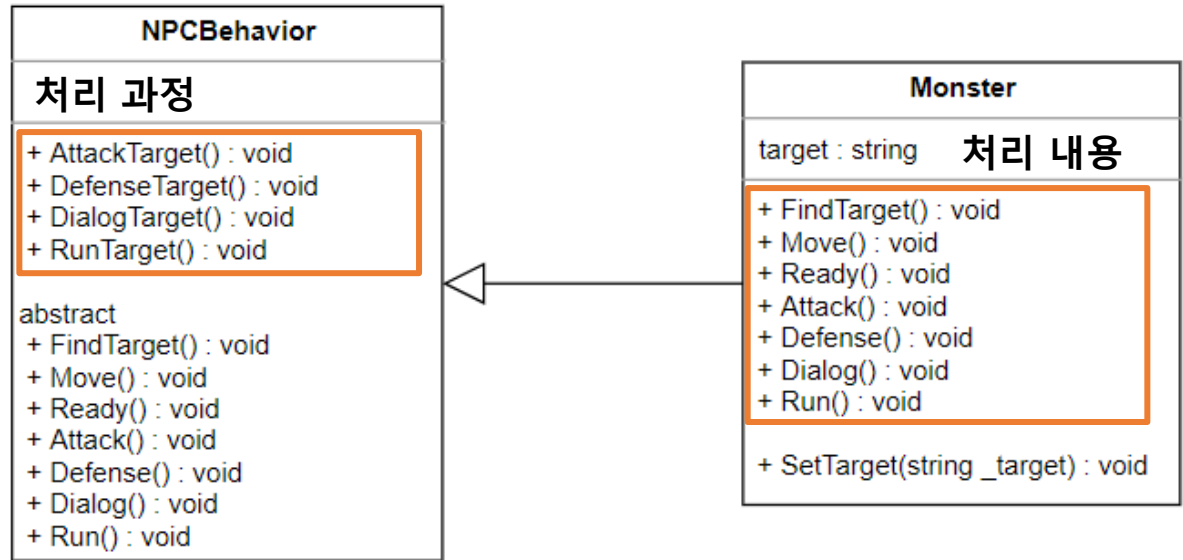
sendData : Interpreter의 역할 과정을 보여준다.

행동 패턴 2 - 템플릿 메서드

템플릿 메서드(template Method)

처리 과정을 부모가, 처리 내용을 자식이 받게 함으로써 메소드를 명확히 하여 일괄적 행동과, 부분적 행동을 포괄적으로 관리하는 패턴

- 상위 클래스에서는 처리과정의 흐름을 제어
- 하위 클래스에서는 처리하는 내용을 제어
- Re-Factoring에 유리한 패턴으로 상속을 통한 확장 개발 방법이다.



<몬스터의 행동을 포괄적으로 관리해주는 프로그램>

디자인 패턴

[목록으로](#)

행동 패턴 2 - 템플릿 메서드

```
class NPCBehavior
{
private:
    virtual void FindTarget() = 0;
    virtual void Move() = 0;
    virtual void Ready() = 0;

    virtual void Attack() = 0;
    virtual void Defense() = 0;
    virtual void Dialog() = 0;
    virtual void Run() = 0;
}
```

```
void DefenseTarget()
{
    FindTarget();
    Move();
    Ready();
    Attack();
}
```

```
void AttackTarget()
{
    FindTarget();
    Move();
    Ready();
    Attack();
}
```

```
void DialogTarget()
{
    FindTarget();
    Move();
    Ready();
    Dialog();
}
```

```
void RunTarget()
{
    FindTarget();
    Move();
    Ready();
    Run();
}
```

NPC의 행동 과정을 일괄적으로 관리하는 메소드



```
#include "common.h"
#include "NPCBehavior.h"

class Monster : public NPCBehavior
{
private:
    string target;

    void FindTarget() { cout << target << "을 찾는다" << endl; }
    void Move() { cout << target << "을 향해 움직인다" << endl; }
    void Ready() { cout << target << "의 앞에서 대기한다" << endl; }

    void Attack() { cout << target << "을 공격한다" << endl; }
    void Defense() { cout << target << "에게 방어한다" << endl; }
    void Dialog() { cout << target << "과 대화한다" << endl; }
    void Run() { cout << target << "에게서 도망간다" << endl; }

public:
    void SetTarget(string _target) { target = _target; }
};
```

몬스터가 행동하는 내역들을 관리하는 실제 몬스터 클래스

```
Monster* monster = new Monster;

monster->SetTarget("주인공");

cout << "—————" << endl;

monster->AttackTarget();

cout << "—————" << endl;

monster->DefenseTarget();

cout << "—————" << endl;

monster->DialogTarget();

cout << "—————" << endl;

monster->RunTarget();

cout << "—————" << endl;
```



다짐이다. 인생의 대기를
그해에 서한다. 차창을
이제까지의 인생을
다져서 만든다.



다짐이다. 인생의 대기를 다
다듬어서 한 해에 겪고
찾아야 할 것이
이제에 있어
더욱더
이것이
자주

[illegible]

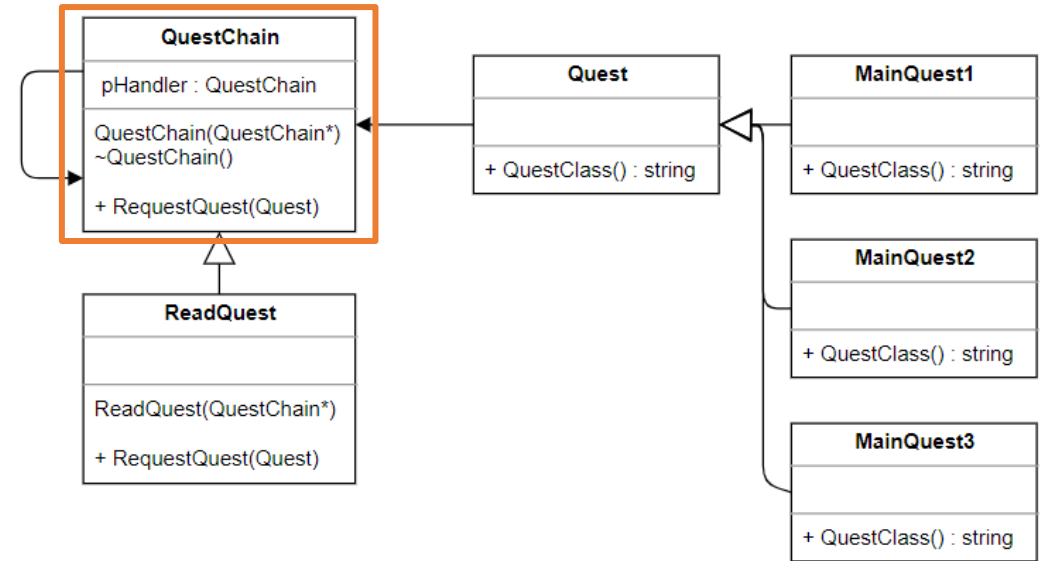
다움 직인다
는해 서 대기한다
찾향 앞
이제에 게서 도망간다
이제에 게서 도망간다
이제에 게서 도망간다

행동 패턴 3 - 책임 연쇄

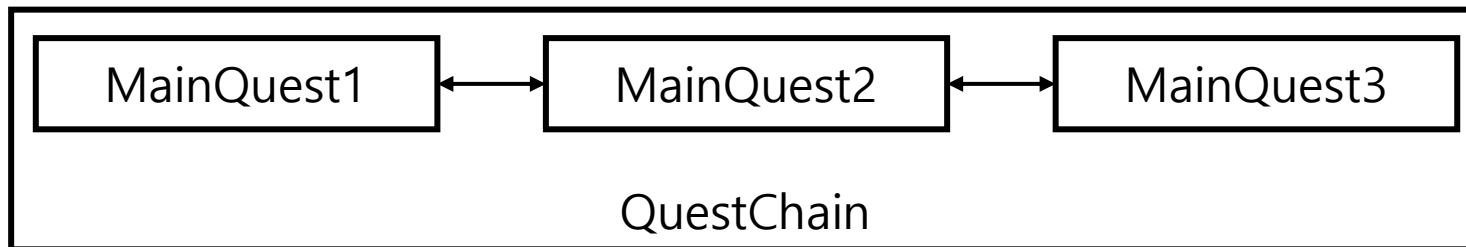
책임 연쇄(Chain of Responsibility)

여러 개의 객체에게 기능을 나눔으로써, 결합도를 나누는 패턴

- 클래스를 리스트로 관리하는 방법
- 처리할 수 있는 모든 객체를 확인한 뒤, 처리하지 못하면 다음 객체로 넘기는 방식



<퀘스트를 단계별로 관리해 줄 수 있는 Chain 프로그램>



호출 하였을 때 불러올 수 있는 Quest를
ReadQuest로 요청한다.

디자인 패턴

목록으로

행동 패턴 3 – 책임 연쇄

```
class Quest
{
public:
    virtual string QuestClass() = 0;
};

class MainQuest1 : public Quest
{
public:
    virtual string QuestClass() {
        return "첫번째 메인 퀘스트";
    }
};

class MainQuest2 : public Quest
{
public:
    virtual string QuestClass() {
        return "두번째 메인 퀘스트";
    }
};

class MainQuest3 : public Quest
{
public:
    virtual string QuestClass() {
        return "세번째 메인 퀘스트";
    }
};
```



담아두는 공간 클래스 생성

```
class QuestChain
{
private:
    QuestChain* pHandler;
public:
    QuestChain(QuestChain* pHandle) : pHandler(pHandle) {}
    ~QuestChain() { if (pHandler) delete pHandler; }

    virtual void RequestQuest(Quest* quest) {
        if (pHandler != NULL) pHandler->RequestQuest(quest);
    }
};
```



```
// quest 넣는 과정 (List push_back과 같은 과정)
QuestChain* quest = new ReadQuest<MainQuest1>(new ReadQuest<MainQuest2>(new ReadQuest<MainQuest3>(NULL)));
```

Quest 리스트를 담는 과정

행동 패턴 3 – 책임 연쇄

```
template <typename T>
class ReadQuest : public QuestChain
{
public:
    ReadQuest(QuestChain* pHandle) : QuestChain(pHandle) {}
    // 기능 내역
    void RequestQuest(Quest* quest) override
    {
        if (quest != NULL)
        {
            cout << quest->QuestClass() << " 진행 중입니다." << endl;
        }
        else
        {
            QuestChain::RequestQuest(quest);
        }
    }
};
```

Template으로 생성자에 사용될 객체 타입을 설정해주어야 한다.

다음 클래스로 넘기는 과정

행동 패턴 3 – 책임 연쇄

퀘스트 요청 소스코드

```
int main()
{
    Quest* firstQuest = new MainQuest1();

    Quest* secondQuest = new MainQuest2();

    Quest* thirdQuest = new MainQuest3();

    // quest 넣는 과정 (List push_back과 같은 과정)
    QuestChain* quest = new ReadQuest<MainQuest1>(new ReadQuest<MainQuest2>(new ReadQuest<MainQuest3>(NULL)));

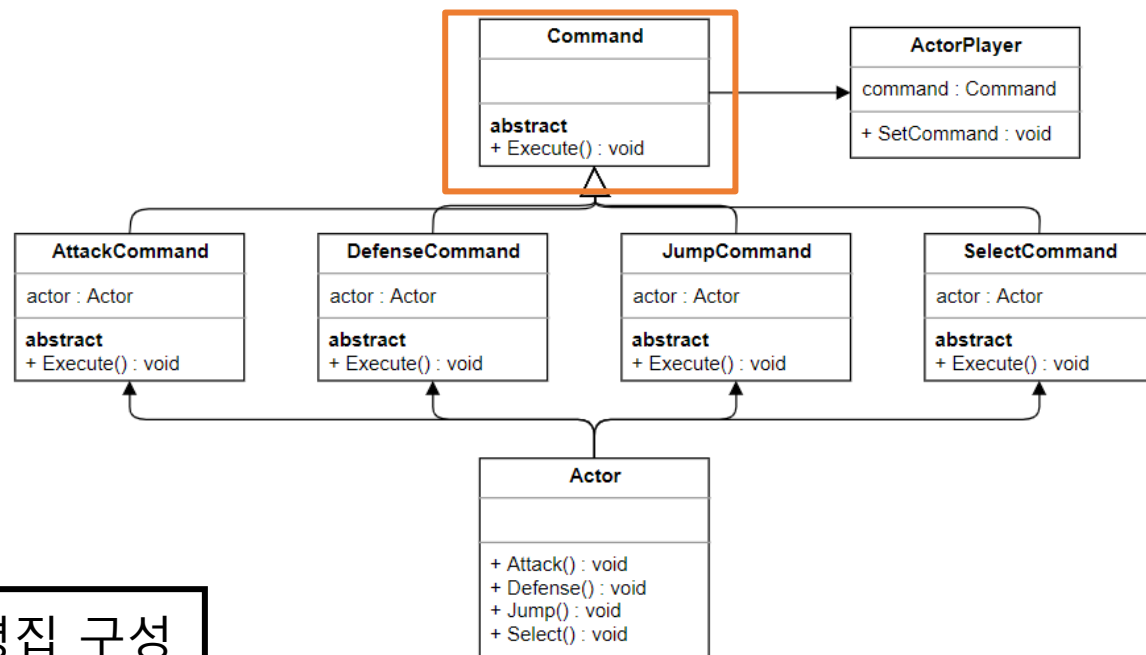
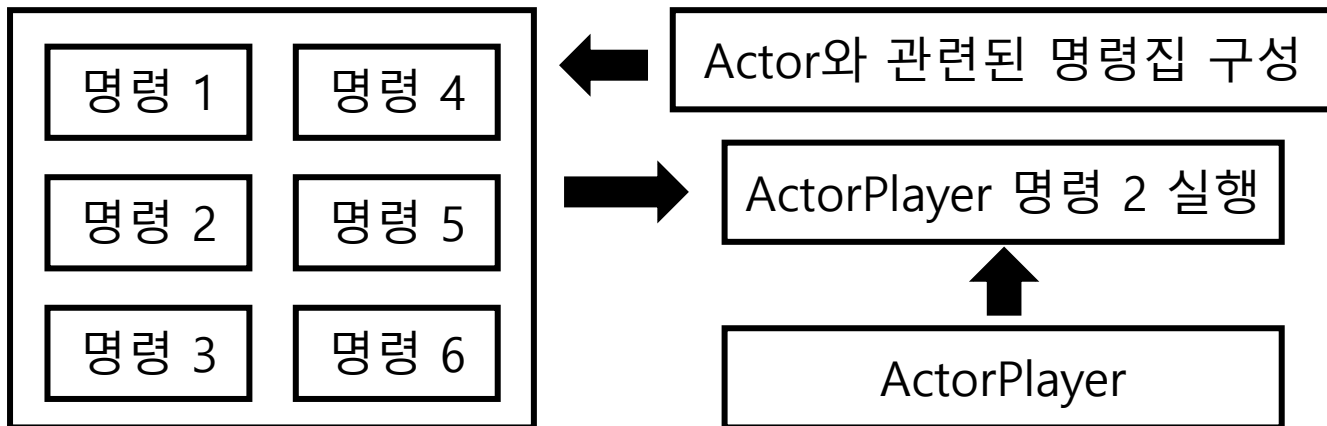
    quest->RequestQuest(firstQuest);
    quest->RequestQuest(secondQuest);
    quest->RequestQuest(thirdQuest);
}
```

행동 패턴 4 – 명령 패턴

명령(Command)

여러가지 행동을 객체로 캡슐화 하여, 사용자 요청시 바로 이용할 수 있도록 구성한 패턴

- 매서드 호출의 실체화
- ActorPlayer->SetCommand(명령)의 형태로 객체화 된 명령을 실행할 수 있다.



<특정 유저의 행동을 관리할 수 있는 프로그램>

디자인 패턴

행동 패턴 4 - 명령 패턴

```
class Actor
{
public:
    void Attack() {
        cout << "공격하다." << endl;
    }
    void Defense() {
        cout << "방어하다." << endl;
    }
    void Jump() {
        cout << "점프하다." << endl;
    }
    void Select() {
        cout << "선택하다." << endl;
    }
};
```

특정 행동을 Command 클래스로 구성한다

```
class AttackCommand : public Command
{
private:
    Actor* actor;
public:
    AttackCommand(Actor* _actor) : actor(_actor){}

    virtual void Execute()
    {
        actor->Attack();
    }
};
```

```
class DefenseCommand : public Command
{
private:
    Actor* actor;
public:
    DefenseCommand(Actor* _actor) : actor(_actor) {}

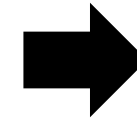
    virtual void Execute()
    {
        actor->Defense();
    }
};
```

```
class JumpCommand : public Command
{
private:
    Actor* actor;
public:
    JumpCommand(Actor* _actor) : actor(_actor) {}

    virtual void Execute()
    {
        actor->Jump();
    }
};
```

```
class SelectCommand : public Command
{
private:
    Actor* actor;
public:
    SelectCommand(Actor* _actor) : actor(_actor) {}

    virtual void Execute()
    {
        actor->Select();
    }
};
```



```
class Command
{
public:
    virtual void Execute() = 0;
};
```

명령을 실제로 실행하는 메소드를 구성한다

디자인 패턴

[목록으로](#)

행동 패턴 4 – 명령 패턴

```
class ActorPlayer
{
private:
    Command* command;
public:
    void SetCommand(Command* _command) {
        command = _command;
        command->Execute();
    }
};
```

실제로 행동하여야 할 플레이어를 구성한다

```
ActorPlayer* player1 = new ActorPlayer();
```

```
Actor* actor = new Actor;
```

```
Command* attack = new AttackCommand(actor);
Command* defense = new DefenseCommand(actor);
Command* select = new SelectCommand(actor);
Command* jump = new JumpCommand(actor);
```



커맨드 객체화

```
player1->SetCommand(attack);
player1->SetCommand(defense);
player1->SetCommand(select);
player1->SetCommand(jump);
```



실제 실행

Actor는 연관된 행동을 모아놓는 역할을 한다.

행동 패턴 5 - 이터레이터

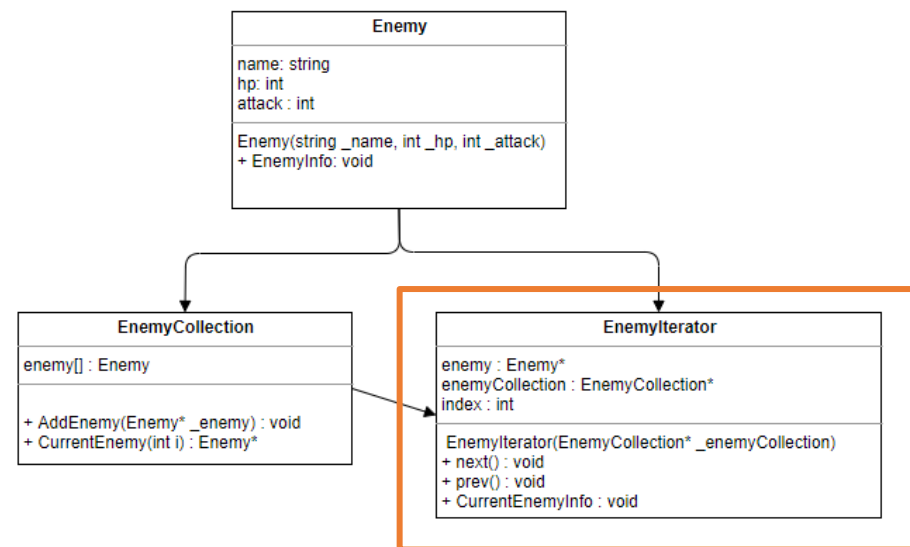
이터레이터(Iterator)

구현한 내용을 분리하여 순차적으로 객체를 관리할 수 있도록 하는 패턴 (반복자)

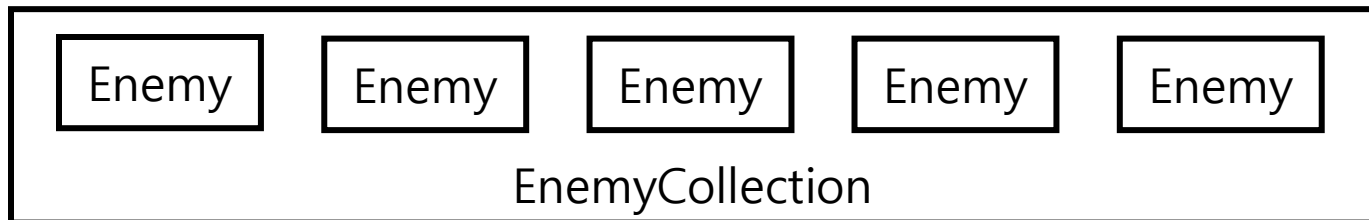
- STL에 자주 사용되는 패턴
- 캡슐화와 은닉화에 초점이 맞춰져 있다.
- 객체 집단을 특정 객체에 넣어 관리하는 방법을 의미한다



접근 가능한 iterator가 없으면 접근이 불가능하다



<적들의 데이터를 Iterator로 관리하는 프로그램>



디자인 패턴

목록으로

행동 패턴 5 - 이터레이터

```
class Enemy
{
private:
    string name;
    int hp;
    int attack;
public:
    void EnemyInfo() {
        cout << "name : " << name << endl;
        cout << "hp : " << hp << endl;
        cout << "attack : " << attack << endl;
    }
    Enemy(string _name, int _hp, int _attack) :name(_name), hp(_hp), attack(_attack){}
};
```

Enemy 모음집에
데이터 넣기



```
class EnemyCollection
{
private:
    Enemy* enemy[10];
public:
    void AddEnemy(Enemy* _enemy)
    {
        for (int i = 0; i < 10; i++)
        {
            if (enemy[i] == NULL)
            {
                enemy[i] = _enemy;
                return;
            }
        }
    }
    Enemy* CurrentEnemy(int i) {
        return enemy[i];
    }
};
```

Iterator에
Enemy / Collection
연결



```
class EnemyIterator
{
private:
    EnemyCollection* enemyCollection;
    int index;
    Enemy* enemy;
public:
    EnemyIterator(EnemyCollection* _enemyCollection) :enemyCollection(_enemyCollection), index(0)
    {
        enemy = enemyCollection->CurrentEnemy(index);
    }
    void next() {
        if (index > 10) return;
        enemy = enemyCollection->CurrentEnemy(++index);
    }
    void prev(){
        if (index < 0) return;
        enemy = enemyCollection->CurrentEnemy(--index);
    }
    void CurrentEnemyInfo()
    {
        enemy->EnemyInfo();
    }
};
```

행동 패턴 5 - 이터레이터

```
EnemyCollection* enemy = new EnemyCollection();

enemy->AddEnemy(new Enemy("고블린1", 10, 10));
enemy->AddEnemy(new Enemy("고블린2", 20, 20));
enemy->AddEnemy(new Enemy("고블린3", 30, 30));
enemy->AddEnemy(new Enemy("고블린4", 40, 40));
enemy->AddEnemy(new Enemy("고블린5", 50, 50));
enemy->AddEnemy(new Enemy("고블린6", 60, 60));
enemy->AddEnemy(new Enemy("고블린7", 70, 70));
enemy->AddEnemy(new Enemy("고블린8", 80, 80));
enemy->AddEnemy(new Enemy("고블린9", 90, 90));
enemy->AddEnemy(new Enemy("히드라", 95, 95));

EnemyIterator* iterator = new EnemyIterator(enemy);

iterator->CurrentEnemyInfo();

iterator->next();

iterator->CurrentEnemyInfo();
```



```
name : 고블린1
hp : 10
attack : 10
name : 고블린2
hp : 20
attack : 20
```

Iterator를 통해 enemy클래스를 불러올 수 있다.

디자인 패턴

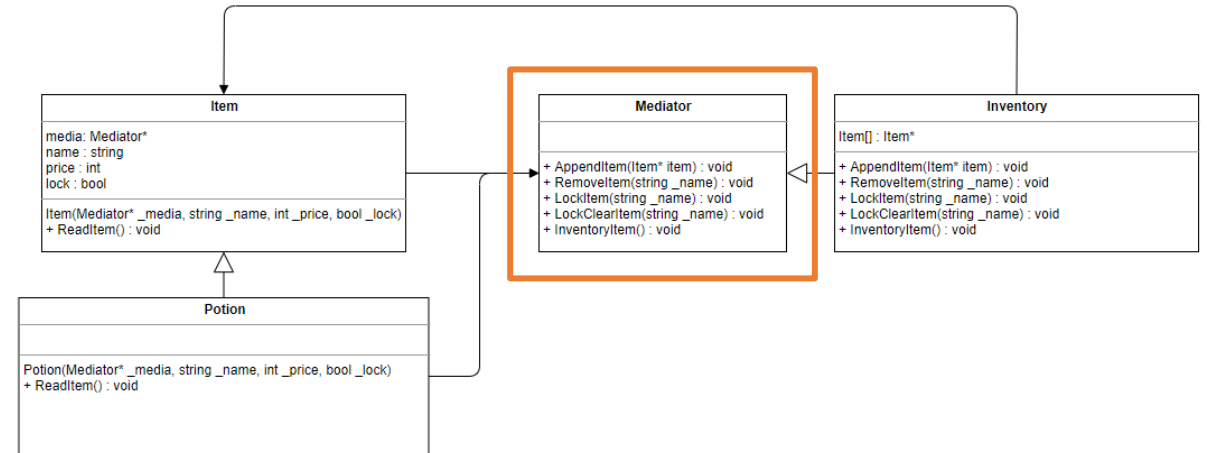
목적으로

행동 패턴 6 - 미디에이터

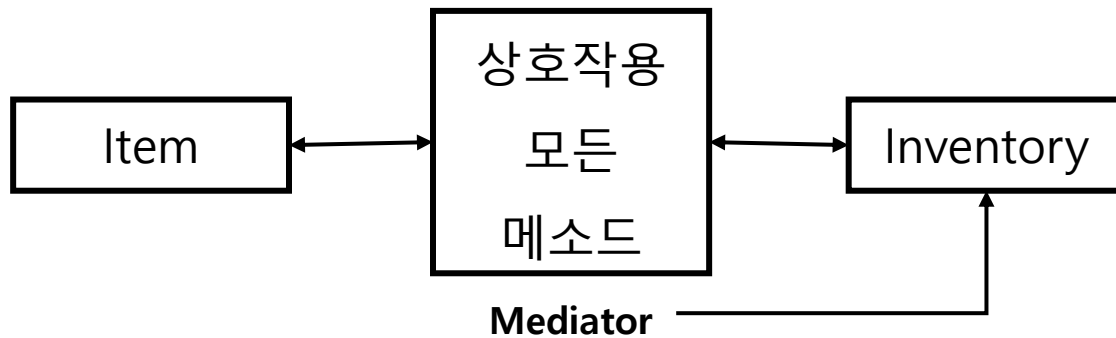
미디에이터(Mediator)

서비스 중개자 패턴

- 객체의 상호작용을 캡슐화하는 객체
- 다대다 관계에서 다대1 관계로 복잡도를 떨어뜨린다.



<인벤토리를 관리하는 중재자 역할이 있는 프로그램>



Mediator 역할을 담당한다

디자인 패턴

[목록으로](#)

행동 패턴 6 - 미디에이터

```
class Item
{
private:
    Mediator* media;
public:
    string name;
    int price;
    bool lock;
public:
    Item(Mediator* _media, string _name, int _price, bool _lock)
        : media(_media), name(_name), price(_price), lock(_lock) {}

    virtual void ReadItem() = 0;
};
```



※ 정방선언에 유의

```
class Item;

class Mediator
{
public:
    virtual void AppendItem(Item* item) = 0;
    virtual void RemoveItem(string _name) = 0;
    virtual void LockItem(string _name) = 0;
    virtual void LockClearItem(string _name) = 0;
    virtual void InventoryItem() = 0;
};
```

```
class Potion : public Item
{
public:
    Potion(Mediator* _media, string _name, int _price, bool _lock)
        : Item(_media, _name, _price, _lock) {}

    virtual void ReadItem() {
        cout << "아이템 이름 : " << name << endl;
        cout << "아이템 가격 : " << price << endl;
        cout << "아이템 잠금 : " << boolalpha << lock << endl;
    }
};
```

<아이템과 관련 있는 클래스 생성>

<중재자 역할을 하는 Mediator>

```
class Inventory : public Mediator
{
private:
    Item* item[50]; // 가방 최대 제한 수를 50개로 규정한다.
                  // 이해를 쉽게 하기 위해 배열로 선정
public:
    // 인벤토리에 들어가는 아이템
    virtual void AppendItem(Item* _item){ ... }

    virtual void InventoryItem(){ ... }

    virtual void RemoveItem(string _name){ ... }

    virtual void LockItem(string _name){ ... }

    virtual void LockClearItem(string _name){ ... }
};
```

디자인 패턴

[목록으로](#)

행동 패턴 6 - 미디에이터

```
Inventory* inventory = new Inventory();

Item* redPotion = new Potion(inventory, "레드 포션", 100, false);
Item* bluePotion = new Potion(inventory, "블루 포션", 500, false);
Item* whitePotion = new Potion(inventory, "하얀 포션", 1000, true);

inventory->AppendItem(redPotion);
inventory->AppendItem(bluePotion);
inventory->AppendItem(whitePotion);

inventory->InventoryItem();
```



```
1 번 아이템 : 레드 포션
아이템 이름 : 레드 포션
아이템 가격 : 100
아이템 잠금 : false

2 번 아이템 : 블루 포션
아이템 이름 : 블루 포션
아이템 가격 : 500
아이템 잠금 : false

3 번 아이템 : 하얀 포션
아이템 이름 : 하얀 포션
아이템 가격 : 1000
아이템 잠금 : true
```

현재 저장된 아이템이 출력되는 것을 확인할 수 있다

```
virtual void AppendItem(Item* item) = 0; // 아이템 추가
virtual void RemoveItem(string _name) = 0; // 아이템 제거
virtual void LockItem(string _name) = 0; // 아이템 락
virtual void LockClearItem(string _name) = 0; // 아이템 락 해제
virtual void InventoryItem() = 0; // 아이템 출력하기
```

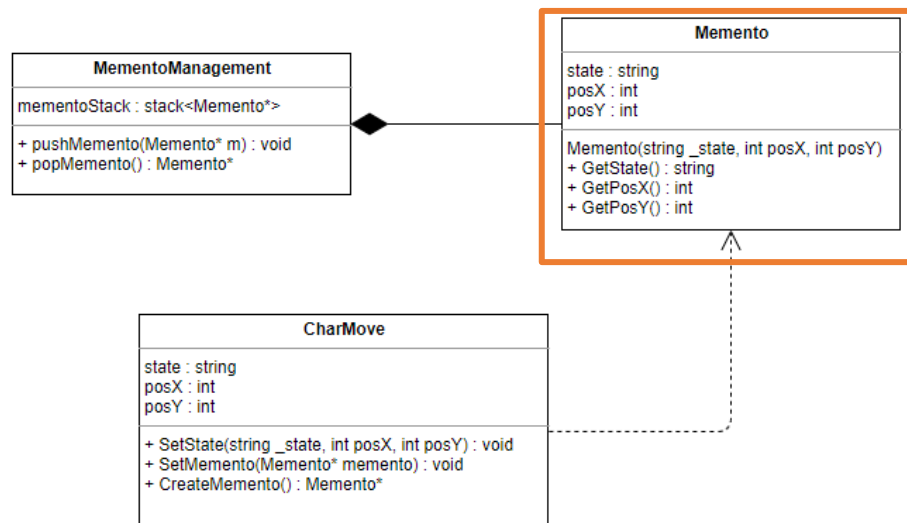
추가적인 기능 설명

행동 패턴 7 - 메멘토

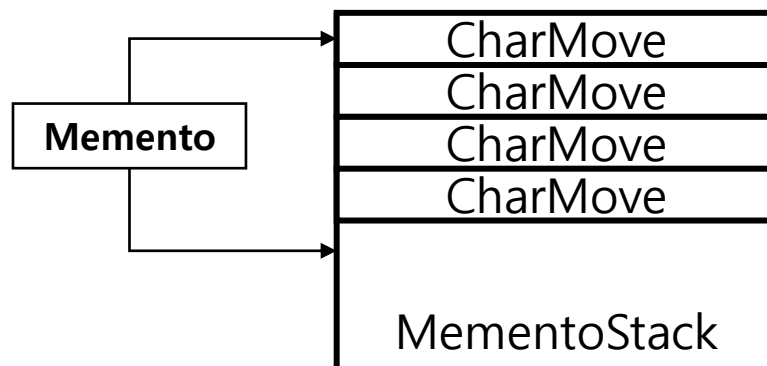
메멘토(Memento)

객체의 상태를 저장 하거나 복원하는 패턴

- 객체 내부의 상태 정보를 저장하고 모아두어 상황에 따라 사용할 수 있다
- 리플레이 : 큐 자료구조를 통해 SAVE/LOAD가 가능하다
- 플레이어 되감기 : 스택 자료구조를 통해 SAVE/LOAD가 가능하다
- 자료가 많아질 수록 문제가 될 수 있다



<플레이어의 움직임을 기록하고 되감기 하는 프로그램>



디자인 패턴

행동 패턴 7 - 메멘토

[목록으로](#)

```
class CharMove
{
private:
    int posX;
    int posY;
    string state;
public:
    void SetState(string _state, int _posX, int _posY) {
        state = _state;
        posX = _posX;
        posY = _posY;
    }

    void SetMemento(Memento* memento) {
        if (memento) {
            state = memento->GetState();
            delete memento;
        }
    }

    Memento* CreateMemento() {
        cout << "상태 : " << state <<
            " X 좌표 : " << posX <<
            " Y 좌표 : " << posY << endl;
        return new Memento(state, posX, posY);
    }
};
```

변수가 같다
= 상태를 저장한다

캐릭터의 움직임을
세팅하는 장소

Memento를 관리하
기 위한 장소

Memento를
(Stack)으로 관리

```
class Memento
{
private:
    string state;
    int posX;
    int posY;
public:
    Memento(string _state, int _posX, int _posY) {
        posX = _posX;
        posY = _posY;
        state = _state;
    }

    string GetState() { return state; }
    int GetPosX() { return posX; }
    int GetPosY() { return posY; }
};
```

```
class MementoManagement
{
private:
    stack<Memento*> mementoStack;
public:
    void pushMemento(Memento* m) {
        mementoStack.push(m);
    }

    Memento* popMemento() {
        Memento* memento = mementoStack.top();
        mementoStack.pop();

        cout << "상태 : " << memento->GetState() <<
            " X 좌표 : " << memento->GetPosX() <<
            " Y 좌표 : " << memento->GetPosY() << endl;

        return memento;
    }
};
```

디자인 패턴

[목록으로](#)

행동 패턴 7 - 메멘토

```
MementoManagement mementoMgr;  
  
CharMove* move = new CharMove();  
  
move->SetState("대기", 10, 10);  
mementoMgr.pushMemento(move->CreateMemento());  
  
move->SetState("대기", 20, 20);  
mementoMgr.pushMemento(move->CreateMemento());  
  
cout << "—————" << endl;  
  
move->SetMemento(mementoMgr.popMemento());  
move->SetMemento(mementoMgr.popMemento());  
  
delete move;
```

캐릭터의 움직임

움직임을 주기적으로 세팅한다

되감기 할 때 pop로 값을 불러온다

대칭되는 결과가 출력된다.

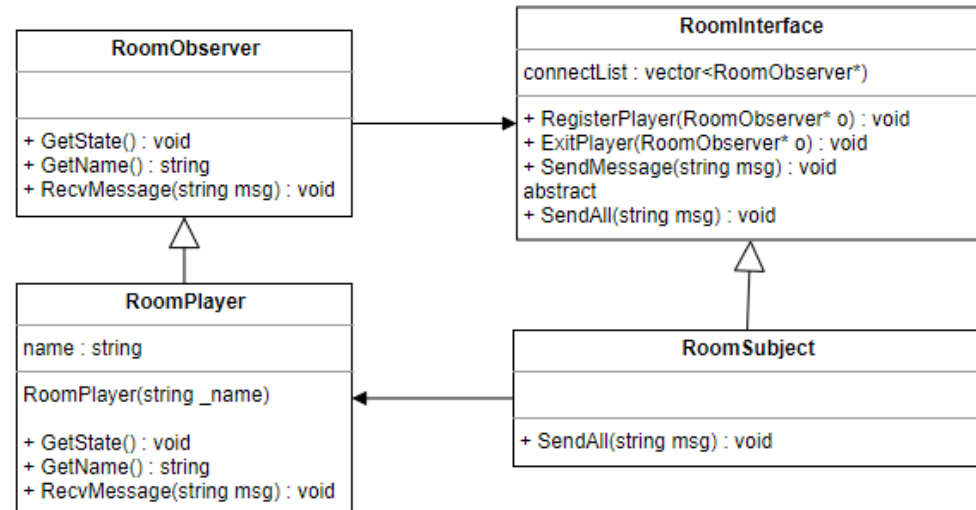
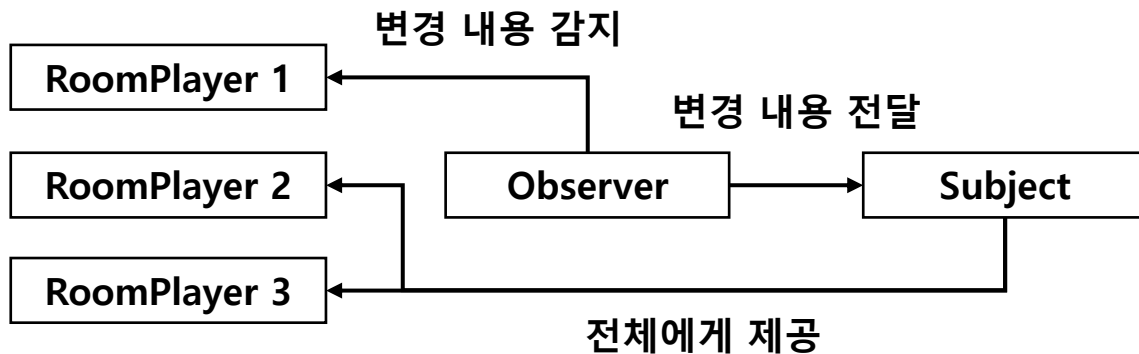
```
상태 : 대기   X 좌표 : 10   Y 좌표 : 10  
상태 : 대기   X 좌표 : 20   Y 좌표 : 20  
-----  
상태 : 대기   X 좌표 : 20   Y 좌표 : 20  
상태 : 대기   X 좌표 : 10   Y 좌표 : 10
```


행동 패턴 8 – 관찰자 패턴

관찰자(Observer)

하나의 객체에 영향에 따라 다른 객체를 함께 변경하는 패턴

- 브로드 캐스트
- 다른 객체가 객체의 갱신 내용을 몰라도 될 경우 사용된다 (예 : 업적 시스템)



<접속자 전체 리스트에 공지를 내려주는 운영 프로그램>

디자인 패턴

목록으로

행동 패턴 8 - 관찰자 패턴

```
class RoomObserver
{
public:
    virtual void GetState() = 0;
    virtual string GetName() = 0;
    virtual void RecvMessage(string msg) = 0;
};

class RoomPlayer : public RoomObserver
{
private:
    string name;
public:
    RoomPlayer(string _name) : name(_name) {};

    void GetState() {
        cout << name << " >> " << endl;
    }
    string GetName() { return name; }

    void RecvMessage(string msg)
    {
        cout << name << " >> " << msg << endl;
    }
};

class RoomSubject : public RoomInterface
{
public:
    void SendAll(string msg) override { SendMessage(msg); }
};
```

가입된 플레이어들을
Vector로 관리한다

사용할 인터페이스를
하나씩 추가해 나간다

```
class RoomInterface
{
private:
    vector<RoomObserver*> connectList;

protected:
    void SendMessage(string msg)
    {
        vector<RoomObserver*>::iterator iter = connectList.begin();
        for (iter = connectList.begin(); iter != connectList.end(); ++iter)
        {
            (*iter)->RecvMessage(msg);
        }
    }

public:
    void RegisterPlayer(RoomObserver* o)
    {
        connectList.push_back(o);

        cout << o->GetName() << "님이 접속하셨습니다" << endl;
    }

    void ExitPlayer(RoomObserver* o)
    {
        vector<RoomObserver*>::iterator iter = connectList.begin();
        for (iter = connectList.begin(); iter != connectList.end(); ++iter)
        {
            if (o->GetName() == (*iter)->GetName())
            {
                cout << o->GetName() << "님이 퇴장하셨습니다" << endl;
                connectList.erase(iter);

                delete (o);
                return;
            }
        }
    }

    virtual void SendAll(string msg) = 0;
};
```

행동 패턴 8 – 관찰자 패턴

```
RoomInterface* room = new RoomSubject();

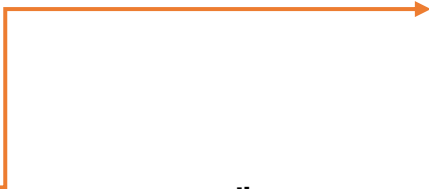
RoomObserver* player1 = new RoomPlayer("유저1");
RoomObserver* player2 = new RoomPlayer("유저2");
RoomObserver* player3 = new RoomPlayer("유저3");

room->RegisterPlayer(player1);
room->RegisterPlayer(player2);
room->RegisterPlayer(player3);

room->SendAll("공지입니다");

room->ExitPlayer(player1);
room->ExitPlayer(player2);
room->ExitPlayer(player3);

delete room;
```



```
유저1님이 접속하셨습니다
유저2님이 접속하셨습니다
유저3님이 접속하셨습니다
유저1 >> 공지입니다
유저2 >> 공지입니다
유저3 >> 공지입니다
유저1님이 퇴장하셨습니다
유저2님이 퇴장하셨습니다
유저3님이 퇴장하셨습니다
```

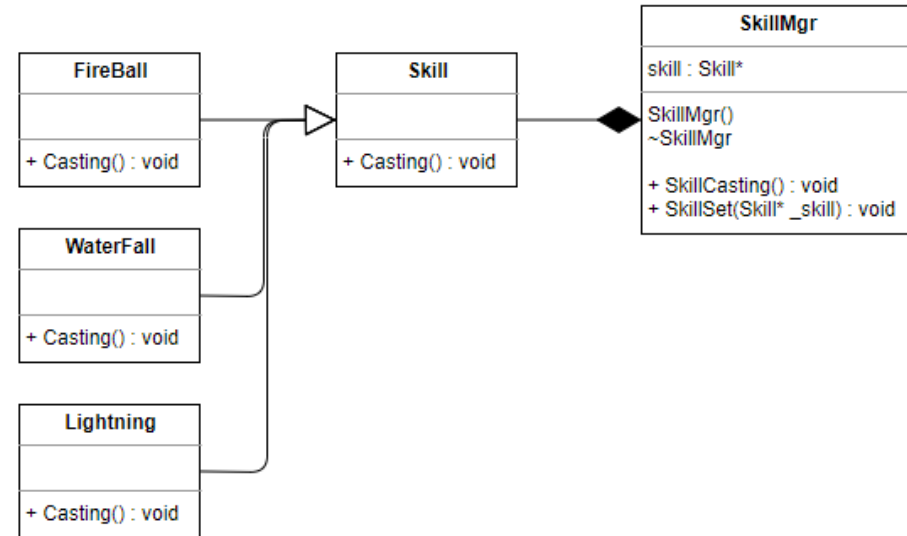
브로드 캐스트로
뿌려주는 것을 확인할
수 있다

행동 패턴 9 – 전략 패턴

전략(Strategy)

알고리즘 및 전략 인터페이스를 동적으로 교체
할 수 있는 패턴

- 알고리즘 인터페이스를 정의하여 이를 클래스별로 캡슐화 하는 방법이다.
- 공통된 목적은 같으나 이를 해결하기 위한 전략 및 알고리즘이 여러가지가 존재할 때 적합한 패턴 (예 : 다양한 정렬 사용)



<스킬 관리 프로그램>

디자인 패턴

[목록으로](#)

행동 패턴 9 – 전략 패턴

```
class Skill
{
public:
    virtual void Casting() = 0;
};

class FireBall : public Skill
{
public:
    void Casting()
    {
        cout << "파이어볼 사용" << endl;
    }
};

class WaterFall : public Skill
{
public:
    void Casting()
    {
        cout << "워터폴 사용 " << endl;
    }
};

class Lightning : public Skill
{
public:
    void Casting()
    {
        cout << "라이트닝 사용 " << endl;
    }
};
```

각각의 스킬을 클래스화
시켜 그 기능을 정의한다



```
class SkillMgr
{
    Skill* skill;
public:
    SkillMgr() : skill(0) {};
    ~SkillMgr() { if (skill) delete skill; }

    void SkillCasting() { skill->Casting(); }

    void SkillSet(Skill* _skill)
    {
        if (skill) delete skill;
        skill = _skill;
    }
};
```

정의한 스킬을 연결해 주는 관리 클래스 설정



```
int main()
{
    SkillMgr* skill = new SkillMgr;

    skill->SkillSet(new FireBall());
    skill->SkillCasting();
    skill->SkillSet(new WaterFall());
    skill->SkillCasting();
    skill->SkillSet(new Lightning());
    skill->SkillCasting();
}
```

같은 클래스로 새로운 클래스를 정의하여 동적
으로 스킬 내용을 바꿀 수 있다.



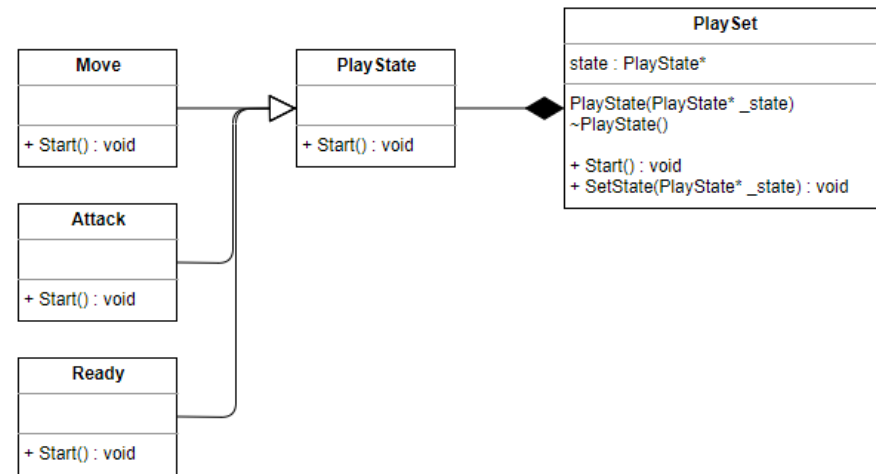
파이어볼 사용
워터폴 사용
라이트닝 사용

행동 패턴 10 – 상태 패턴

상태(State)

객체의 상태를 객체화 하여 그 행위를 변경할 수 있도록 캡슐화 한 패턴

- 전략 패턴과 구조가 동일하나 목적이 다르다.
- 동적으로 상황에 따라 행동을 교체할 수 있다.



<플레이어의 행동을 관리하는 프로그램>

디자인 패턴

목록으로

행동 패턴 10 – 상태 패턴

```
class PlayState
{
public:
    virtual void Start() = 0;
};

class Move : public PlayState
{
public:
    void Start(){
        cout << "플레이어 이동을 시작합니다." << endl;
    }
};

class Attack : public PlayState
{
public:
    void Start() {
        cout << "공격을 시작합니다." << endl;
    }
};

class Ready : public PlayState
{
public:
    void Start() {
        cout << "대기합니다." << endl;
    }
};
```

각각의 행동을 클래스화
시켜 그 기능을 정의한다



```
class PlaySet
{
private:
    PlayState* state;
public:
    PlaySet(PlayState* _state) : state(_state) {}
    ~PlaySet() { if (state) delete state; }

    void SetState(PlayState* _state) { if (state) delete state; state = _state; }
    void Start() { state->Start(); }
};
```

정의한 행동을 연결해 주는 관리 클래스 설정



같은 클래스로 새로운 클래스를 정의하여 동적
으로 스킬 내용을 바꿀 수 있다.

※ 전략 패턴과 유사하므로
사용 목적을 확실히 하여
사용하여야 한다

```
PlaySet* play = new PlaySet(new Ready());
play->Start();

play->SetState(new Move());
play->Start();

play->SetState(new Attack());
play->Start();

delete play;
```

디자인 패턴

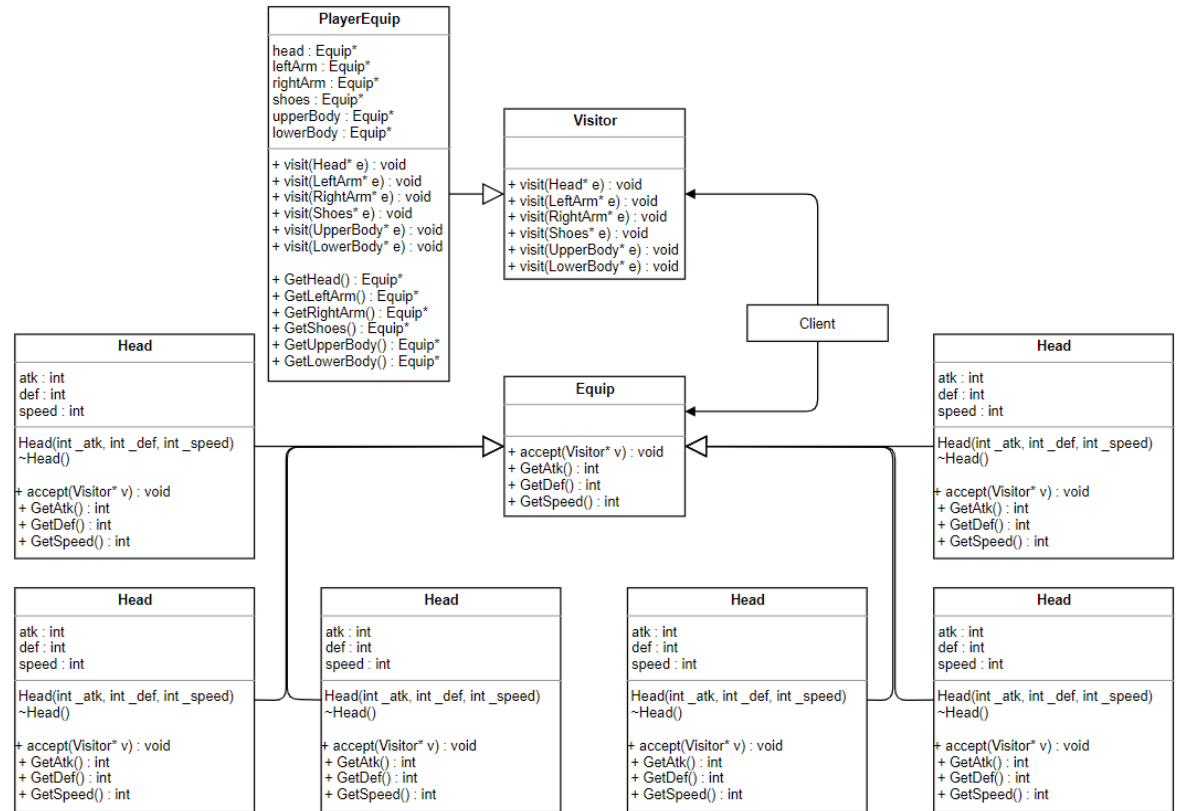
목록으로

행동 패턴 11 – 방문자 패턴

방문자(Visitor)

객체가 가지고 있는 구조와 기능을 분리시키는 패턴

- 여러개의 기능들 중 하나의 기능을 따로 추가, 변경, 삭제 하여야 할 경우 사용되는 패턴
- Visitor : 객체로 부터 분리된 기능들을 모아 놓은 장소
- Equip(Element) : 객체로 부터 분리된 기능 및 요소들을 의미



<플레이어의 장비를 세팅하는 프로그램>

디자인 패턴

[목록으로](#)

행동 패턴 11 – 방문자 패턴

```
class Head : public Equip
{
    int atk;
    int def;
    int speed;
public:
    Head(int _atk, int _def, int _speed) : atk(_atk), def(_def), speed(_speed) {};
    ~Head() {};
    virtual void accept(Visitor* v) { v->visit(this); };

    virtual int GetAtk() {return atk;}
    virtual int GetDef() { return def; }
    virtual int GetSpeed() { return speed; }
};
```

```
class LeftArm : public Equip
{
    int atk;
    int def;
    int speed;
public:
    LeftArm(int _atk, int _def, int _speed) : atk(_atk), def(_def), speed(_speed) {};
    ~LeftArm() {};
    virtual void accept(Visitor* v) { v->visit(this); };

    virtual int GetAtk() { return atk; }
    virtual int GetDef() { return def; }
    virtual int GetSpeed() { return speed; }
};
```

```
class RightArm : public Equip
{
    int atk;
    int def;
    int speed;
public:
    RightArm(int _atk, int _def, int _speed) : atk(_atk), def(_def), speed(_speed) {};
    ~RightArm() {};
    virtual void accept(Visitor* v) { v->visit(this); };

    virtual int GetAtk() { return atk; }
    virtual int GetDef() { return def; }
    virtual int GetSpeed() { return speed; }
};
```

```
class Equip
{
public:
    virtual void accept(Visitor* v) = 0;
    virtual int GetAtk() = 0;
    virtual int GetDef() = 0;
    virtual int GetSpeed() = 0;
};
```

```
class Shoes : public Equip
{
    int atk;
    int def;
    int speed;
public:
    Shoes(int _atk, int _def, int _speed) : atk(_atk), def(_def), speed(_speed) {};
    ~Shoes() {};
    virtual void accept(Visitor* v) { v->visit(this); };

    virtual int GetAtk() { return atk; }
    virtual int GetDef() { return def; }
    virtual int GetSpeed() { return speed; }
};
```

```
class UpperBody : public Equip
{
    int atk;
    int def;
    int speed;
public:
    UpperBody(int _atk, int _def, int _speed) : atk(_atk), def(_def), speed(_speed) {};
    ~UpperBody() {};
    virtual void accept(Visitor* v) { v->visit(this); };

    virtual int GetAtk() { return atk; }
    virtual int GetDef() { return def; }
    virtual int GetSpeed() { return speed; }
};
```

```
class LowerBody : public Equip
{
    int atk;
    int def;
    int speed;
public:
    LowerBody(int _atk, int _def, int _speed) : atk(_atk), def(_def), speed(_speed) {};
    ~LowerBody() {};
    virtual void accept(Visitor* v) { v->visit(this); };

    virtual int GetAtk() { return atk; }
    virtual int GetDef() { return def; }
    virtual int GetSpeed() { return speed; }
};
```

플레이어의
장비 요소
분리

디자인 패턴

목록으로

행동 패턴 11 – 방문자 패턴

```
class Visitor
{
public:
    virtual void visit(Head* e) = 0;
    virtual void visit(LeftArm* e) = 0;
    virtual void visit(RightArm* e) = 0;
    virtual void visit(Shoes* e) = 0;
    virtual void visit(UpperBody* e) = 0;
    virtual void visit(LowerBody* e) = 0;
};
```



분리된 요소가 방문하는 인터페이스
Visitor



```
class PlayerEquip : public Visitor
{
    Equip* head;
    Equip* leftArm;
    Equip* rightArm;
    Equip* shoes;
    Equip* upperBody;
    Equip* lowerBody;
public:
    virtual void visit(Head* e) { if (head) { delete head; } head = e; }
    virtual void visit(LeftArm* e) { if (leftArm) { delete leftArm; } leftArm = e; };
    virtual void visit(RightArm* e) { if (rightArm) { delete rightArm; } rightArm = e; };
    virtual void visit(Shoes* e) { if (shoes) { delete shoes; } shoes = e; };
    virtual void visit(UpperBody* e) { if (upperBody) { delete upperBody; } upperBody = e; };
    virtual void visit(LowerBody* e) { if (lowerBody) { delete lowerBody; } lowerBody = e; };

    Equip* GetHead() {
        cout << "<머리 장비 스탯> 공격 : " << head->GetAtk() <<
            " 방어: " << head->GetDef() <<
            " 속도: " << head->GetSpeed() << endl;
        return head; }
    Equip* GetLeftArm() { ... }
    Equip* GetRightArm() { ... }
    Equip* GetShoes() { ... }
    Equip* GetUpperBody() { ... }
    Equip* GetLowerBody() { ... }
};
```

디자인 패턴

목록으로

행동 패턴 11 – 방문자 패턴

```
class Visitor
{
public:
    virtual void visit(Head* e) = 0;
    virtual void visit(LeftArm* e) = 0;
    virtual void visit(RightArm* e) = 0;
    virtual void visit(Shoes* e) = 0;
    virtual void visit(UpperBody* e) = 0;
    virtual void visit(LowerBody* e) = 0;
};
```



분리된 요소가 방문하는 인터페이스
Visitor



```
class PlayerEquip : public Visitor
{
    Equip* head;
    Equip* leftArm;
    Equip* rightArm;
    Equip* shoes;
    Equip* upperBody;
    Equip* lowerBody;
public:
    virtual void visit(Head* e) { if (head) { delete head; } head = e; }
    virtual void visit(LeftArm* e) { if (leftArm) { delete leftArm; } leftArm = e; };
    virtual void visit(RightArm* e) { if (rightArm) { delete rightArm; } rightArm = e; };
    virtual void visit(Shoes* e) { if (shoes) { delete shoes; } shoes = e; };
    virtual void visit(UpperBody* e) { if (upperBody) { delete upperBody; } upperBody = e; };
    virtual void visit(LowerBody* e) { if (lowerBody) { delete lowerBody; } lowerBody = e; };

    Equip* GetHead() {
        cout << "<머리 장비 스탯> 공격 : " << head->GetAtk() <<
            " 방어: " << head->GetDef() <<
            " 속도: " << head->GetSpeed() << endl;
        return head; }
    Equip* GetLeftArm() { ... }
    Equip* GetRightArm() { ... }
    Equip* GetShoes() { ... }
    Equip* GetUpperBody() { ... }
    Equip* GetLowerBody() { ... }
};
```

디자인 패턴

[목록으로](#)

행동 패턴 11 – 방문자 패턴

```
PlayerEquip* player = new PlayerEquip();

player->visit(new Head(10, 10, 10));
player->visit(new LeftArm(20, 20, 20));
player->visit(new RightArm(30, 30, 30));
player->visit(new Shoes(40, 40, 40));
player->visit(new LowerBody(50, 50, 50));
player->visit(new UpperBody(60, 60, 60));

cout << "Player가 착용중인 장비 " << endl;
player->GetHead();
player->GetLeftArm();
player->GetRightArm();
player->GetShoes();
player->GetLowerBody();
player->GetUpperBody();
```



```
Player가 착용중인 장비
<머리> 장비 스텟> 공격력 : 10 방어 : 10 속도 : 10
<왼팔> 장비 스텟> 공격력 : 20 방어 : 20 속도 : 20
<오른팔> 장비 스텟> 공격력 : 30 방어 : 30 속도 : 30
<신발> 장비 스텟> 공격력 : 40 방어 : 40 속도 : 40
<하의> 장비 스텟> 공격력 : 50 방어 : 50 속도 : 50
<상의> 장비 스텟> 공격력 : 60 방어 : 60 속도 : 60
```

Player에게 장비를 이어주었을 시 출력 내용

행동 패턴 최종 정리

행동 패턴 [Behavior Patten]	알고리즘을 어느 객체에 적용할지에 대한 구조를 담은 패턴	구성 영역
인터프리터 [Interpreter]	문법 규칙을 클래스화, 일련의 규칙으로 정의된 언어 해석 패턴	클래스 단위
템플릿 메서드 [Template Method]	처리 과정을 부모가, 처리 내용을 자식, 메소드를 명확히 하여 일괄적 행동과 부분적 행동을 포괄적으로 처리하는 패턴	클래스 단위
책임 연쇄 [Chain of Responsibility]	여러 개의 객체에게 기능을 나눔으로써, 결합도를 나누는 패턴 (클래스의 리스트화)	객체 단위
명령 [Command]	여러가지 행동을 객체로 캡슐화, 사용자 요청 시 바로 이용할 수 있도록 구성한 패턴	객체 단위
반복자 [Iterator]	구현한 내용을 분리하여 순차적으로 객체를 관리할 수 있도록 하는 패턴	객체 단위
중재자 [Mediator]	중간단계에서 각 객체의 메소드를 중개해주는 역할을 담당하는 패턴	객체 단위
메멘토 [Memento]	객체의 상태를 저장 및 복원 하는 패턴	객체 단위
관찰자 [Observer]	한 객체의 정보를 전체에게 공지하는데 사용되는 패턴	객체 단위
전략 [Strategy]	알고리즘 및 전략 인터페이스를 동적으로 관리할 수 있는 패턴	객체 단위
상태 [State]	객체의 상태를 객체화 하여 그 행위를 변경할 수 있도록 캡슐화 한 패턴	객체 단위
방문자 [Visitor]	객체가 가지고 있는 구조와 기능을 분리시키는 패턴	객체 단위

디자인 패턴

참고 문헌

- [1] Erich Gamma is co-author of Design Patterns: Elements of Reusable Object-Oriented Software
- [2] JAVA 객체 지향 디자인 패턴 – 한빛 미디어
- [3] 게임 프로그래밍 패턴 – 한빛 미디어
- [4] Cechich, Alejandra, and Richard Moore. "A formal specification of GoF design patterns." *Proceedings of the Asia Pacific Software Engineering Conference: APSEC*. Vol. 99. 1999.
- [5] Cooper, James W. *C# Design patterns: A tutorial*. Addison-Wesley Professional, 2002.

[EX-1] 소년 코딩 블로그 - <https://boycoding.tistory.com/>

[EX-2] Thinking Difference - <https://copynull.tistory.com/>

[EX-2] UML작성 사이트 - <https://app.diagrams.net/>

공유 파일

<https://drive.google.com/file/d/16RKQb3OcH1tWWz-Od8i9ZBJ7hwjuPY84/view?usp=sharing> // 디자인 패턴 1부

<https://drive.google.com/file/d/1uT-1jN-EcD4j6JJzAnfXhsAVqKu6W5XC/view?usp=sharing> // 디자인 패턴 2부